

6B21ICSC: Introduction to Deep Learning

Lecture Notes

Shyma P V

LaTeX Prepared by Abhinav Raj

Contents

I Unit I

1 Introduction to Deep Learning 11

1.1 What is Deep Learning ? 11

1.1.1 Deep Learning Vs Machine Learning 12

1.1.2 Difference Between Machine Learning and Deep Learning 12

1.2 What are Neural Networks ? 13

1.2.1 The Basic Architecture of Artificial Neural Network 15

1.2.2 Perceptron 16

1.2.3 Single Layer Perceptron 17

1.2.4 Multi-layer Perceptron 18

1.2.5 Structure of MultiLayer Perceptron Neural Network 19

1.2.6 The basic building blocks of deep learning 19

1.3 Fairness, Accountability, and Transparency in Machine Learning 20

1.3.1 Fairness 20

1.3.2 Accountability 20

1.3.3 Transparency 21

1.4 Applications of Deep Learning 21

1.4.1 Computer vision 21

1.4.2 Natural language processing (NLP) 21

1.4.3 Reinforcement learning 22

II Unit II

2 Foundations of Deep Learning 25

2.1 Introduction 25

2.2 Activation Function 25

2.2.1 Why do we need Non-linear activation function? 26

2.3 Key Activation Functions and Their Applicability 27

2.3.1 Binary step function 27
2.3.2 Linear Function 28
2.3.3 Sigmoid Function 28
2.3.4 Tanh Function 31
2.3.5 RELU Function 31
2.3.6 Softmax Function 32
2.3.7 Leaky ReLU Function 32
2.3.8 Swish Function 32
2.3.9 Choosing Activation Functions for Neural Networks 34

2.4 Capacity of Model 35

2.4.1 Overfitting 35
2.4.2 Underfitting 35

2.5 Hyperparameter 36

2.5.1 Hyperparameters in Simple Neural Networks 36

2.6 Data Splitting: Training, Testing, and Validation Datasets 37

2.6.1 Training Dataset 37
2.6.2 Validation Dataset 37
2.6.3 Test Dataset 37

2.7 Estimator 38

2.7.1 Uses of Estimators 38
2.7.2 Classification of Estimators 38
2.7.3 Bias and Variance 38

2.8 Deep Networks 39

2.8.1 Feed forward networks 39

2.9 Working Principle of a Feed Forward Neural Network 39

2.9.1 Single-Layer Perceptron 39

2.10 Layers in a Feed Forward Neural Network 40

2.10.1 Input Layer 40
2.10.2 Output Layer 40
2.10.3 Hidden Layer 40

2.11 Key Components 40

2.11.1 Neuron Weights 40
2.11.2 Neurons 40
2.11.3 Activation Function 41
2.11.4 Function in feed forward neural network 41
2.11.5 Learning XOR 41

2.12 The XOR Problem Solution **41**
2.12.1 Introduction to XOR Problem . . . 41
2.12.2 Limitation of Single-Layer Perceptron . . . 42
2.12.3 Graphical Representation of XOR Problem . . . 42
2.12.4 Limitation of Single-Layer Perceptron . . . 42
2.12.5 Solution Using Multilayer Perceptron . . . 43
2.13 Gradient-Based Learning **44**
2.13.1 Gradient Descent Algorithm . . . 44
2.13.2 Loss Function . . . 44
2.13.3 Gradient Computation . . . 44
2.14 Hidden Units – Architecture Design **45**
2.14.1 Definition and Role of Hidden Units . . . 45
2.14.2 Weight Updation in Hidden Units . . . 45
2.14.3 Diagram: Neural Network Architecture . . . 46
2.14.4 Equations and Activation Functions . . . 46
2.15 Backpropagation Algorithm **46**
2.15.1 Forward Pass . . . 46
2.15.2 Loss Calculation . . . 46
2.15.3 Backward Pass . . . 47
2.15.4 Weight Update . . . 47
2.15.5 Example and Diagram . . . 47
2.15.6 Back propagation . . . 48
2.16 Compute Error **49**
2.17 Backward Pass **49**
2.17.1 Differentitation in Deep learning . . . 50

III **Unit III**

3 Regularization Techniques in Deep Learning . . . 53
3.1 Regularization Strategies and Techniques **53**
3.1.1 Common Regularization Techniques . . . 53
3.2 Loss Function Modification: Adding Regularization Terms **54**
3.3 Parameter Norm Penalties **54**
3.4 L2 Regularization **55**
3.4.1 Loss Function with L2 Regularization . . . 55
3.4.2 Interpretation of the Penalty Term . . . 55
3.4.3 Effect on Model Training . . . 55
3.4.4 Gradient Descent with L2 Regularization . . . 56
3.4.5 Choosing the Regularization Parameter α . . . 56
3.4.6 Example . . . 56
3.4.7 Summary . . . 56
3.5 L1 Regularization **56**
3.5.1 Loss Function with L1 Regularization . . . 57
3.5.2 Interpretation of the Penalty Term . . . 57
3.5.3 Effect on Model Training . . . 57
3.5.4 Gradient Descent with L1 Regularization . . . 57

3.5.5 Choosing the Regularization Parameter α . . . 58
3.5.6 Example . . . 58
3.5.7 Summary . . . 58
3.6 Norm Penalties as Constrained Optimization 58
3.6.1 Constrained Optimization Framework . . . 58
3.6.2 Equivalence of Penalized and Constrained Forms . . . 59
3.6.3 Interpretation and Benefits . . . 59
3.6.4 Example: Linear Regression with L2 Regularization . . . 60
3.6.5 Example: Linear Regression with L1 Regularization . . . 60
3.7 Regularized & Under-constrained Problems 60
3.7.1 Importance of Data Segmentation . . . 61
3.8 Noise Robustness 61
3.8.1 Benefits of Noise Robustness . . . 61
3.9 Semi-Supervised Learning 62
3.10 Multi-Task Learning 62
3.11 Early Stopping of Training 62
3.11.1 How Early Stopping Works . . . 63
3.11.2 Example . . . 63
3.12 Bagging 63
3.13 Dropout 64
3.14 Adversarial Training 64
3.15 Parameter Tying and Sharing 64
3.16 Sparse Representations 65
3.17 Tangent Distance, Tangent Prop, and Manifold Tangent Classifier 65
3.17.1 Understanding Manifold Space . . . 65
3.18 Tangent Distance for Regularization 66
3.18.1 Tangent Distance . . . 66
3.19 Tangent Propagation for Robustness 66
3.19.1 Conceptual Framework . . . 67
3.20 Manifold Tangent Classifier for Improved Classification 67
3.20.1 Key Components . . . 67
3.20.2 Implementation Steps . . . 67

IV Unit IV

4 Optimization for Training Deep Models . . . 71
4.1 Understanding the Difference: Learning vs. Pure Optimization 71
4.1.1 Machine Learning Perspective . . . 71
4.1.2 Pure Optimization Angle . . . 71
4.1.3 Understanding Empirical Risk Minimization (ERM) . . . 71
4.1.4 Expected Risk . . . 72

4.2 Visualizing the Differences 72
4.3 Challenges in Neural Network Optimization 72
4.3.1 Ill-conditioning . . . 72
4.3.2 Local Minima . . . 74
4.3.3 Plateaus, Saddle Points, and Other Flat Regions . . . 74
4.3.4 Cliffs and Exploding Gradients . . . 75
4.3.5 Long-Term Dependencies . . . 76
4.3.6 Weight Space Symmetry . . . 76
4.4 Basic Optimization Algorithms 76
4.4.1 Gradient Descent Algorithm . . . 76
4.4.2 Stochastic Gradient Descent (SGD) . . . 77
4.4.3 Adam . . . 77
4.4.4 Momentum . . . 77
4.5 Parameter Initialization Strategies 78
4.6 Algorithms with Adaptive Learning Rates 78
4.6.1 AdaGrad . . . 78
4.6.2 RMSProp . . . 78
4.6.3 Adam . . . 79
4.7 Approximate Second-Order Methods in Deep Learning 79
4.7.1 Gauss-Newton Method . . . 79
4.7.2 Limited-Memory BFGS (L-BFGS) . . . 79
4.7.3 Natural Gradient Descent . . . 79
4.8 Optimization Strategies and Meta-Algorithms 79
4.8.1 Optimization Strategies . . . 80
4.8.2 Meta-Algorithms . . . 80

Unit I

1 Introduction to Deep Learning 11
1.1 What is Deep Learning ?
1.2 What are Neural Networks ?
1.3 Fairness, Accountability, and Transparency in Machine Learning
1.4 Applications of Deep Learning

1. Introduction to Deep Learning

1.1 What is Deep Learning ?

Deep learning is a branch of machine learning based on artificial neural networks. It is a method in artificial intelligence (AI) that teaches computers to process data in a way inspired by the human brain. Deep learning models can recognize complex patterns in pictures, text, sounds, and other data to produce accurate insights and predictions.

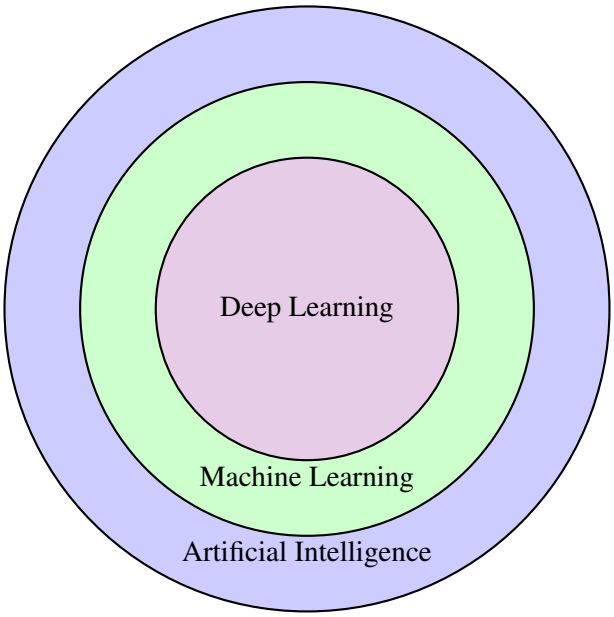
It has grown in popularity in recent years as computing power has increased and massive datasets have become more widely available. Deep learning relies on artificial neural networks (ANNs), also known as deep neural networks (DNNs). These neural networks are modeled after the structure and function of actual neurons in the human brain and are intended to learn from vast amounts of data.

- Deep Learning is an area of Machine Learning that uses neural networks to model and solve complicated problems. Neural networks are modeled after the structure and function of the human brain, with layers of interconnected nodes processing and transforming input.
- Deep Learning's distinguishing feature is the usage of deep neural networks, which consist of numerous layers of interconnected nodes. These networks may learn sophisticated representations of data by identifying hierarchical patterns and characteristics. Deep Learning systems can automatically learn and improve from data, eliminating the need for manual feature engineering.
- Deep Learning has made important advances in a variety of domains, including image identification, natural language processing, speech recognition, and recommendation systems. CNNs, RNNs, and DBNs are some of the most common Deep Learning architectures.
- Deep neural networks are often trained using a significant quantity of data and computer resources. However, the availability of cloud computing and the development of specialised technology, such as graphics processing units (GPUs), has made it easier to train deep neural networks.

1.1.1 Deep Learning Vs Machine Learning

Both machine learning and deep learning have the potential to transform a wide range of industries, including healthcare, finance, retail, and transportation, by providing insights and automating decision-making processes.

- **Machine Learning**: Machine learning is a subset, an application of Artificial Intelligence (AI) that offers the ability of the system to learn and improve from experience without being programmed to that level. Machine Learning uses data to train and find accurate results. Machine learning focuses on the development of a computer program that accesses the data and uses it to learn from itself.
- **Deep Learning**: Deep Learning is a subset of Machine Learning where the artificial neural network and the recurrent neural network come in relation. The algorithms are created exactly just like machine learning but it consists of many more levels of algorithms. All these networks of the algorithm are together called the artificial neural network. In much simpler terms, it replicates just like the human brain as all the neural networks are connected in the brain, which exactly is the concept of deep learning. It solves all the complex problems with the help of algorithms and its process.



1.1.2 Difference Between Machine Learning and Deep Learning

SNo.	Machine Learning	Deep Learning
1	Machine Learning is a superset of Deep Learning	Deep Learning is a subset of Machine Learning
2	The data represented in Machine Learning is quite different compared to Deep Learning as it uses structured data	The data representation used in Deep Learning is quite different as it uses neural networks (ANN).
3	Machine Learning is an evolution of AI.	Deep Learning is an evolution of Machine Learning. Basically, it is how deep is the machine learning.
4	Machine learning consists of thousands of data points.	Big Data: Millions of data points.
5	Outputs: Numerical Value, like classification of the score.	Anything from numerical values to free-form elements, such as free text and sound.

6	Uses various types of automated algorithms that turn to model functions and predict future action from data.	Uses a neural network that passes data through processing layers to interpret data features and relations.
7	Algorithms are detected by data analysts to examine specific variables in data sets.	Algorithms are largely self-depicted on data analysis once they're put into production.
8	Machine Learning is highly used to stay in the competition and learn new things.	Deep Learning solves complex machine-learning issues.
9	Training can be performed using the CPU (Central Processing Unit).	A dedicated GPU (Graphics Processing Unit) is required for training.
10	More human intervention is involved in getting results.	Although more difficult to set up, deep learning requires less intervention once it is running Deep Learning solves complex machine-learning issues.
11	Machine learning systems can be swiftly set up and run, but their effectiveness may be constrained.	Although they require additional setup time, deep learning algorithms can produce results immediately (although the quality is likely to improve over time as more data becomes available).
12	Its model takes less time in training due to its small size.	A huge amount of time is taken because of very big data points.
13	Humans explicitly do feature engineering.	Feature engineering is not needed because important features are automatically detected by neural networks.
14	Machine learning applications are simpler compared to deep learning and can be executed on standard computers.	Deep learning systems utilize much more powerful hardware and resources.
15	The results of an ML model are easy to explain.	The results of deep learning are difficult to explain.
16	Machine learning models can be used to solve straightforward or a little bit challenging issues.	Deep learning models are appropriate for resolving challenging issues.
17	Banks, doctor's offices, and mailboxes all employ machine learning already.	Deep learning technology enables increasingly sophisticated and autonomous algorithms, such as self-driving automobiles or surgical robots.
18	Machine learning involves training algorithms to identify patterns and relationships in data.	Deep learning, on the other hand, uses complex neural networks with multiple layers to analyze more intricate patterns and relationships.
19	Machine learning algorithms can range from simple linear models to more complex models such as decision trees and random forests.	Deep learning algorithms, on the other hand, are based on artificial neural networks that consist of multiple layers and nodes.
20	Machine learning algorithms typically require less data than deep learning algorithms, but the quality of the data is more important.	Deep learning algorithms, on the other hand, require large amounts of data to train the neural networks but can learn and improve on their own as they process more data.
21	Machine learning is used for a wide range of applications, such as regression, classification, and clustering.	Deep learning, on the other hand, is mostly used for complex tasks such as image and speech recognition, natural language processing, and autonomous systems.
22	Machine learning algorithms for complex tasks, but they can also be more difficult to train and may require more computational resources.	Deep learning algorithms are more accurate than machine learning algorithms.

1.2 What are Neural Networks ?

A neural network is a method in artificial intelligence that teaches computers to process data in a way that is inspired by the human brain. It is a type of machine learning process, called

deep learning, that uses interconnected nodes or neurons in a layered structure that resembles the human brain. It creates an adaptive system that computers use to learn from their mistakes and improve continuously. Neural networks can help computers make intelligent decisions with limited human assistance. This is because they can learn and model the relationships between input and output data that are nonlinear and complex. For instance, they can do the following tasks. **Make generalizations and inferences** Neural networks can comprehend unstructured data and make general observations without explicit training. **W**hat are neural networks used for? Neural networks have several use cases across many industries, such as the following: Medical diagnosis by medical image classification Targeted marketing by social network filtering and behavioral data analysis Financial predictions by processing historical data of financial instruments Electrical load and energy demand forecasting Process and quality control Chemical compound identification We give four of the important applications of neural networks below. **Computer vision** Computer vision is the ability of computers to extract information and insights from images and videos. With neural networks, computers can distinguish and recognize images similar to humans. Computer vision has several applications, such as the following:

→ Visual recognition in self-driving cars so they can recognize road signs and other road users
→ Content moderation to automatically remove unsafe or inappropriate content from image and video archives
→ Facial recognition to identify faces and recognize attributes like open eyes, glasses, and facial hair
→ Image labeling to identify brand logos, clothing, safety gear, and other image details

Speech recognition

Neural networks can analyze human speech despite varying speech patterns, pitch, tone, language, and accent. Virtual assistants like Amazon Alexa and automatic transcription software use speech recognition to do tasks like these:

- Assist call center agents and automatically classify calls
- Convert clinical conversations into documentation in real time
- Accurately subtitle videos and meeting recordings for wider content reach

Natural language processing

Natural language processing (NLP) is the ability to process natural, human-created text. Neural networks help computers gather insights and meaning from text data and documents. NLP has several use cases, including in these functions:

- Automated virtual agents and chatbots
- Automatic organization and classification of written data
- Business intelligence analysis of long-form documents like emails and forms
- Indexing of key phrases that indicate sentiment, like positive and negative comments on social media
- Document summarization and article generation for a given topic

Recommendation engines

Neural networks can track user activity to develop personalized recommendations. They can also analyze all user behavior and discover new products or services that interest a specific user. For example, Curalate, a Philadelphia-based startup, helps brands convert social media posts into sales. Brands use Curalate's intelligent product tagging (IPT) service to automate the collection and curation of user-generated social content. IPT uses neural networks to automatically find and recommend products relevant to the user's social media activity. Consumers don't have to hunt through online catalogs to find a specific product from a social media image. Instead, they can use Curalate's auto product tagging to purchase the product with ease.

1.2.1 The Basic Architecture of Artificial Neural Network

The term "Artificial neural network" refers to a biologically inspired sub-field of artificial intelligence modeled after the brain. An Artificial neural network is usually a computational network based on biological neural networks that construct the structure of the human brain. Similar to a human brain has neurons interconnected to each other, artificial neural networks also have neurons that are linked to each other in various layers of the networks. These neurons are known as nodes.

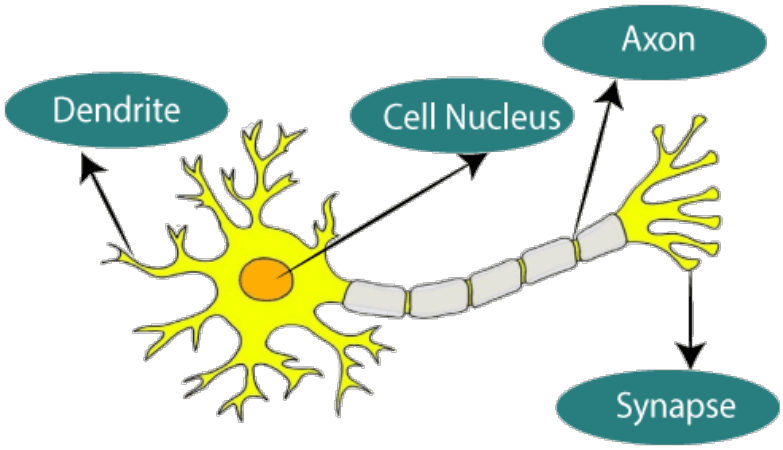


Figure 1.1: Biological Neural Network

Dendrites from Biological Neural Network represent inputs in Artificial Neural Networks, cell nucleus represents Nodes, synapse represents Weights, and Axon represents Output.

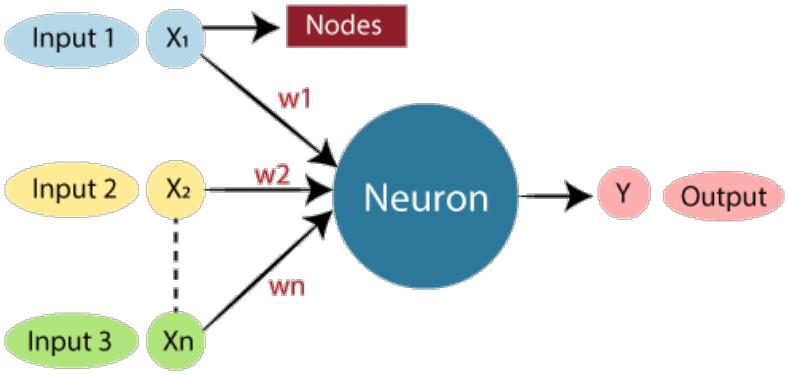


Figure 1.2: Artificial Neural Network

To understand the concept of the architecture of an artificial neural network, we have to understand what a neural network consists of. In order to define a neural network that consists of a large number of artificial neurons, which are termed units arranged in a sequence of layers.

Input Layer: As the name suggests, it accepts inputs in several different formats provided by the programmer.
Hidden Layer: The hidden layer presents in-between input and output layers. It performs all the calculations to find hidden features and patterns.
Output Layer: The input goes through a series of transformations using the hidden layer, which finally results in output that is conveyed using this layer.

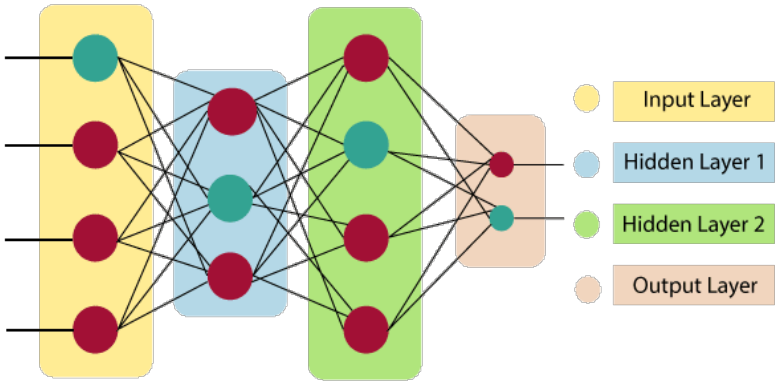


Figure 1.3: The Basic Architecture of Artificial Neural Network

The artificial neural network takes input and computes the weighted sum of the inputs and includes a bias. This computation is represented in the form of a transfer function.

$$\sum_{i=1}^{n} W_i X_i + b$$

It determines weighted total is passed as an input to an activation function to produce the output. Activation functions choose whether a node should fire or not. Only those who are fired make it to the output layer.

1.2.2 Perceptron

The perceptron is a single processing unit of any neural network. Frank Rosenblatt first proposed in 1958 is a simple neuron which is used to classify its input into one or two categories. The original Perceptron was designed to take a number of binary inputs, and produce one binary output (0 or 1). The idea was to use different weights to represent the importance of each input, and that the sum of the values should be greater than a threshold value before making a decision like yes or no (true or false) (0 or 1). As perceptron is mainly used to classify the data into two parts. Therefore, it is also known as **Linear Binary Classifier**.

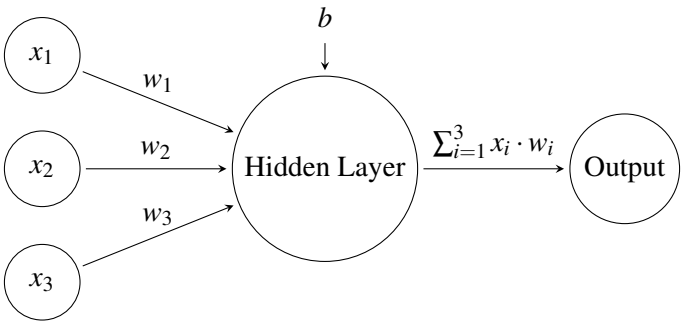


Figure 1.4: Neural network architecture.

Perceptron uses the step function that returns +1 if the weighted sum of its input 0 and -1. The activation function is used to map the input between the required value like (0, 1) or (-1, 1).

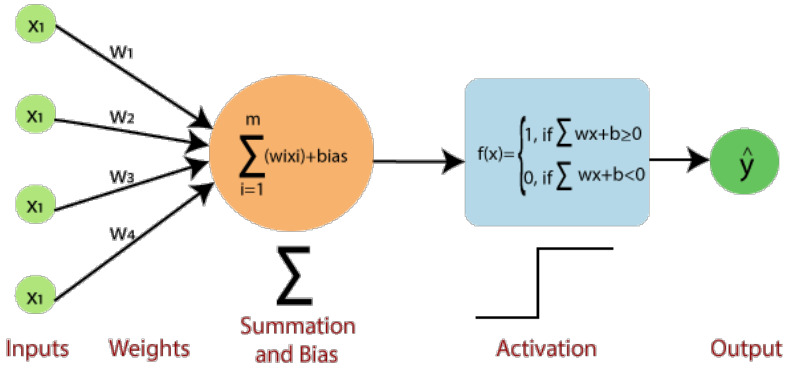


Figure 1.5: perceptron

The perceptron consists of 4 parts.
Input value or One input layer: The input layer of the perceptron is made of artificial input neurons and takes the initial data into the system for further processing.
Weight: It represents the dimension or strength of the connection between units. If the weight to node 1 to node 2 has a higher quantity, then neuron 1 has a more considerable influence on the neuron.
Bias: It is the same as the intercept added in a linear equation. It is an additional parameter which task is to modify the output along with the weighted sum of the input to the other neuron.
Net sum: It calculates the total sum.
Activation Function: A neuron can be activated or not, is determined by an activation function. The activation function calculates a weighted sum and further adding bias with it to give the result.

1.2.3 Single Layer Perceptron

The single-layer perceptron was the first neural network model, proposed in 1958 by Frank Rosenbluth. It is one of the earliest models for learning. Our goal is to find a linear decision function measured by the weight vector w and the bias parameter b.

To understand the perceptron layer, it is necessary to comprehend artificial neural networks (ANNs).

The **Artificial Neural Network** (ANN) is an information processing system, whose mechanism is inspired by the functionality of biological neural circuits. An artificial neural network consists of several processing units that are interconnected.

The single vector perceptron is calculated by calculating the sum of the input vector multiplied by the corresponding element of the vector, with each increasing the amount of the corresponding component of the vector by weight. The value that is displayed in the output is the input of an activation function.

The weights are initialized with the random values at the origination of each training.

For each element of the training set, the error is calculated with the difference between the desired output and the actual output. The calculated error is used to adjust the weight.

The process is repeated until the fault made on the entire training set is less than the specified limit until the maximum number of iterations has been reached.

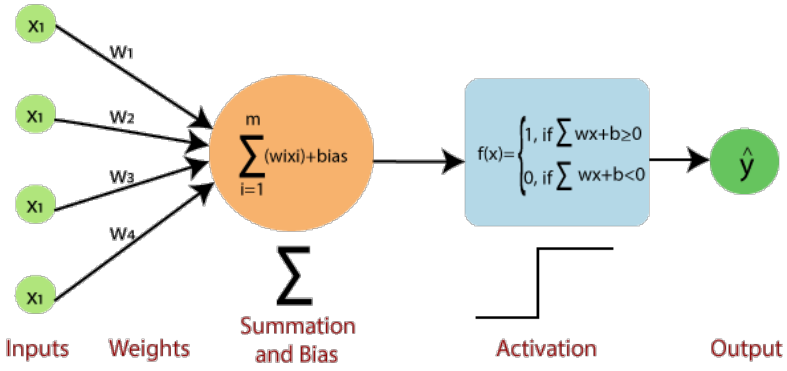


Figure 1.6: Singlelayer perceptron

1.2.4 Multi-layer Perceptron

Multi-layer perception is also known as MLP. It is fully connected dense layers, which transform any input dimension to the desired dimension. A multi-layer perception is a neural network that has multiple layers. To create a neural network we combine neurons together so that the outputs of some neurons are inputs of other neurons.

A multi-layer perceptron has one input layer and for each input, there is one neuron(or node), it has one output layer with a single node for each output and it can have any number of hidden layers and each hidden layer can have any number of nodes. A schematic diagram of a Multi-Layer Perceptron (MLP) is depicted below.

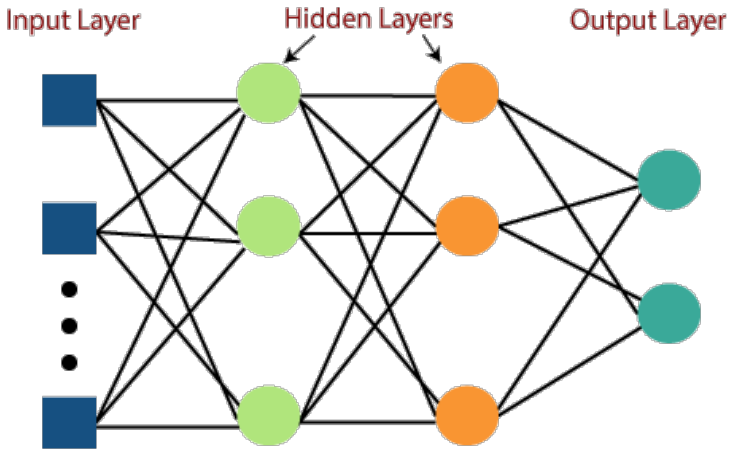


Figure 1.7: Multi-layer perceptron

In the multi-layer perceptron diagram above, we can see that there are a number of inputs and thus number of input nodes and the hidden layer has number of nodes. The output layer gives two outputs, therefore there are two output nodes.

The nodes in the input layer take input and forward it for further process, in the diagram above the nodes in the input layer forwards their output to each of the three nodes in the hidden layer, and in the same way, the hidden layer processes the information and passes it to the output layer.

1.2.5 Structure of MultiLayer Perceptron Neural Network

This netwok has three main layers that combine to form a complete Artificial Neural Network. These layers are as follows:

Input Layer : It is the initial or starting layer of the Multilayer perceptron. It takes input from the training data set and forwards it to the hidden layer. There are n input nodes in the input layer. The number of input nodes depends on the number of dataset features. Each input vector variable is distributed to each of the nodes of the hidden layer.

Hidden Layer : It is the heart of all Artificial neural networks. This layer comprises all computations of the neural network. The edges of the hidden layer have weights multiplied by the node values. This layer uses the activation function.

There can be one or two hidden layers in the model.

Several hidden layer nodes should be accurate as few nodes in the hidden layer make the model unable to work efficiently with complex data. More nodes will result in an overfitting problem.

Output Layer : This layer gives the estimated output of the Neural Network. The number of nodes in the output layer depends on the type of problem. For a single targeted variable, use one node. N classification problem, ANN uses N nodes in the output layer.

Working of MultiLayer Perceptron Neural Network

- The input node represents the feature of the dataset.
- Each input node passes the vector input value to the hidden layer.
- In the hidden layer, each edge has some weight multiplied by the input variable. All the production values from the hidden nodes are summed together. To generate the output
- The activation function is used in the hidden layer to identify the active nodes.
- The output is passed to the output layer.
- Calculate the difference between predicted and actual output at the output layer.
- The model uses backpropagation after calculating the predicted output.

Advantages of MultiLayer Perceptron Neural Network

- MultiLayer Perceptron Neural Networks can easily work with non-linear problems.
- It can handle complex problems while dealing with large datasets.
- Developers use this model to deal with the fitness problem of Neural Networks.
- It has a higher accuracy rate and reduces prediction error by using backpropagation.
- After training the model, the Multilayer Perceptron Neural Network quickly predicts the output.

Disadvantages of MultiLayer Perceptron Neural Network

- This Neural Network consists of large computation, which sometimes increases the overall cost of the model.
- The model will perform well only when it is trained perfectly.
- Due to this model's tight connections, the number of parameters and node redundancy increases.

1.2.6 The basic building blocks of deep learning

Deep Learning is a type of Machine Learning that is based on the use of neural networks. Neural networks are computational systems that can learn from data and perform tasks such as classification, prediction, and decision-making.

Neural networks are the building blocks of Deep Learning. They are computational systems that can learn from data and perform a wide range of tasks. Understanding the basics of neural networks, including neurons, layers, weights, biases, and training, is crucial to understanding the principles of Deep Learning (figure :1.2).

Neural networks are a class of computational algorithms that mimic the functioning of the human brain. They are based on the idea of interconnected processing nodes, called neurons, which work together to process and analyze data. A neural network consists of an input layer, one or more hidden layers, and an output layer. Each layer contains one or more neurons, which are connected to neurons in the next layer by a set of weights.

Neurons

The basic unit of a neural network is a neuron. A neuron receives input from other neurons or from an external source and produces an output based on that input. The output of a neuron is determined by its activation function, which is a mathematical function that maps the input to an output. The activation function can be a simple threshold function, a sigmoid function, a ReLU function, or any other function that is appropriate for the problem at hand.

Layers

A neural network consists of multiple layers of neurons. The input layer is the first layer of the network, which receives input data from an external source. The output layer is the last layer of the network, which produces the final output of the network. The hidden layers are the layers in between the input and output layers, which perform computations on the input data.

Weights and Bias

Each connection between neurons in adjacent layers is associated with a weight. The weight determines the strength of the connection between the neurons. The weights are updated during the training process of the neural network to minimize the error between the predicted output and the actual output. In addition to weights, each neuron in a neural network has a bias term, which is a constant value that is added to the input of the neuron. The bias term allows the neuron to adjust its output independently of the input.

Training

The process of training a neural network involves adjusting the weights and biases of the neurons to minimize the error between the predicted output and the actual output. This is typically done using an optimization algorithm such as stochastic gradient descent. During training, the neural network learns to recognize patterns in the input data and make predictions based on those patterns.

1.3 Fairness, Accountability, and Transparency in Machine Learning

1.3.1 Fairness

in machine learning is the process of correcting algorithmic bias in automated decision processes. This includes bias based on race, ethnicity, gender, sexual orientation, disability, and class.

Fairness is a formalization of the minimization of disparate treatment and impact. It's a recently established area of machine learning that studies how to ensure that biases in the data and model inaccuracies do not lead to models that treat individuals unfavorably.

Some strategies to ensure fairness in machine learning include: Regularization, Adversarial learning, Post-processing.

1.3.2 Accountability

in machine learning refers to the degree to which humans can oversee and correct algorithms. It also refers to who is responsible and culpable if problems arise.

According to the OECD, accountability means that organizations and individuals are expected to ensure that the AI systems they develop, design, operate, or deploy function properly. This should be done in accordance with their roles and applicable regulatory frameworks.

The algorithm's creator could be held accountable if there's an error in the algorithm. This error could lead to a stable deviation or unpredictable behavior, causing erroneous outcomes.

1.3.3 Transparency

in machine learning (ML) is the degree to which users understand how decisions are made and that ML is used to automate decisions. It's also about how much users can understand about the inner workings of a system.

Transparent ML is when the model explains itself, including how it works, its predictions, and insights. This helps users understand and trust the outcome.

Transparency is important for maintaining human agency and building trust in the system. For example, if an algorithm is used to determine whether someone is accepted for a loan, that person should be able to see why the decision was made. This allows them to take actions to change the decision in the future.

Transparency also means that the logic, assumptions, data, and outcomes of ML models are clear and understandable to stakeholders. This includes developers, users, regulators, and affected communities.

1.4 Applications of Deep Learning

The main applications of deep learning can be divided into computer vision, natural language processing (NLP), and reinforcement learning.

1.4.1 Computer vision

In computer vision, Deep learning models can enable machines to identify and understand visual data. Some of the main applications of deep learning in computer vision include:

Object detection and recognition : Deep learning model can be used to identify and locate objects within images and videos, making it possible for machines to perform tasks such as self-driving cars, surveillance, and robotics.

Image classification : Deep learning models can be used to classify images into categories such as animals, plants, and buildings. This is used in applications such as medical imaging, quality control, and image retrieval.

Image segmentation : Deep learning models can be used for image segmentation into different regions, making it possible to identify specific features within images.

1.4.2 Natural language processing (NLP)

In NLP, the Deep learning model can enable machines to understand and generate human language. Some of the main applications of deep learning in NLP include:

Automatic Text Generation : Deep learning model can learn the corpus of text and new text like summaries, essays can be automatically generated using these trained models.

Language translation : Deep learning models can translate text from one language to another, making it possible to communicate with people from different linguistic backgrounds.

Sentiment analysis : Deep learning models can analyze the sentiment of a piece of text, making it possible to determine whether the text is positive, negative, or neutral. This is used in applications such as customer service, social media monitoring, and political analysis.

Speech recognition : Deep learning models can recognize and transcribe spoken words, making it possible to perform tasks such as speech-to-text conversion, voice search, and voice-controlled devices.

1.4.3 Reinforcement learning

In reinforcement learning, deep learning works as training agents to take action in an environment to maximize a reward. Some of the main applications of deep learning in reinforcement learning include:

Game playing : Deep reinforcement learning models have been able to beat human experts at games such as Go, Chess, and Atari.

Robotics : Deep reinforcement learning models can be used to train robots to perform complex tasks such as grasping objects, navigation, and manipulation.

Control systems : Deep reinforcement learning models can be used to control complex systems such as power grids, traffic management, and supply chain optimization.

Unit II

2 Foundations of Deep Learning 25

2.1 Introduction
2.2 Activation Function
2.3 Key Activation Functions and Their Applicability
2.4 Capacity of Model
2.5 Hyperparameter
2.6 Data Splitting: Training, Testing, and Validation Datasets
2.7 Estimator
2.8 Deep Networks
2.9 Working Principle of a Feed Forward Neural Network
2.10 Layers in a Feed Forward Neural Network
2.11 Key Components
2.12 The XOR Problem Solution
2.13 Gradient-Based Learning
2.14 Hidden Units – Architecture Design
2.15 Backpropagation Algorithm
2.16 Compute Error
2.17 Backward Pass

2. Foundations of Deep Learning

2.1 Introduction

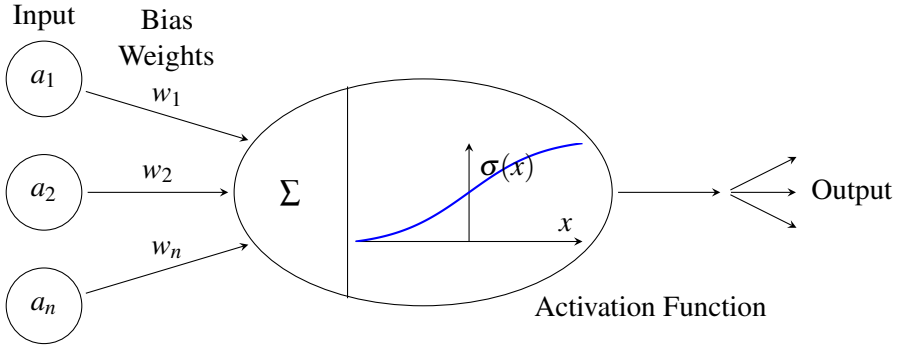
The neural network's input layer receives data, such as images, audio, or text, for training. Input features are weighted and biased to compute a linear function, whose output serves as input for the activation function, producing activations for subsequent layers.

In Convolutional Neural Networks (CNNs), where complexity is high, researchers have explored various methods to improve performance, including different image classification techniques and regularization methods like Dropout. Amid these efforts, the activation function remains crucial, influencing CNN effectiveness across domains such as object recognition, speech analysis, and autonomous driving.

The choice of activation function is vital during implementation. This article will explore different activation functions, analyzing their strengths and weaknesses.

2.2 Activation Function

The activation function of a node in an artificial neural network is a function that calculates the output of the node based on its individual inputs and their weights. Nontrivial problems can be solved using only a few nodes if the activation function is nonlinear.



Basically, three important steps took place in a single iteration of deep neural architectures: **For-ward Propagation**, **Backward Propagation**, and **Gradient Descent(Optimization)**.

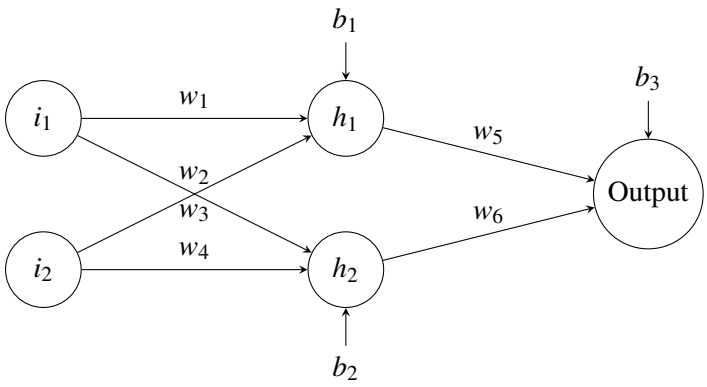
1. **Forward Propagation**: Input data is passed forward through the network, undergoing linear transformations at each layer, followed by activation function application.
2. **Back Propagation**: This step involves calculating derivatives of the loss function with respect to the network's parameters, enabling parameter updates for optimization.
3. **Optimization**: Various optimization algorithms, such as **Gradient Descent**, **Momentum**, **Adam**, and others, are employed to iteratively update the parameters, minimizing the loss function and enhancing model performance.

2.2.1 Why do we need Non-linear activation function?

A neural network without an activation function is essentially just a linear regression model. The activation function does the non-linear transformation to the input making it capable to learn and perform more complex tasks.

Mathematical proof

Suppose we have a Neural net like this :-



Elements of the diagram are as follows :-

Hidden layer i.e. layer 1:

$$z(1) = W(1)X + b(1) = a(1)$$

Here,

1. $z(1)$ is the vectorized output of layer 1
2. $W(1)$ be the vectorized weights assigned to neurons of hidden layer i.e. w_1,w_2,w_3,w_4
3. X be the vectorized input features i.e. i_1 and i_2
4. b is the vectorized bias assigned to neurons in hidden layer i.e. b_1 and b_2
5. $a(1)$ is the vectorized form of any linear function.

(Note: We are not considering activation function here)

Output layer i.e. Layer 2 :

Note : Input for layer 2 is output from layer 1

$$z(2) = W(2)a(1) + b(2)$$

$$a(2) = z(2)$$

Calculation at Output layer

$$z(2) = (W(2) * [W(1)X + b(1)]) + b(2)$$

$$z(2) = [W(2) * W(1)] * X + [W(2) * b(1) + b(2)]$$

Let,

$$[W(2) * W(1)] = W$$

$$[W(2) * b(1) + b(2)] = b$$

Final output : $z(2) = W * X + b$ which is again a linear function

This observation results again in a linear function even after applying a hidden layer, hence we can conclude that, doesn't matter how many hidden layer we attach in neural net, all layers will behave same way because the composition of two linear function is a linear function itself. Neu-ron cannot learn with just a linear function attached to it. A non-linear activation function will let it learn as per the difference w.r.t error. Hence we need an activation function.

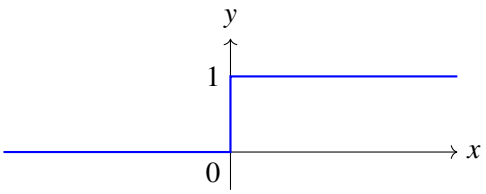
2.3 Key Activation Functions and Their Applicability

2.3.1 Binary step function

The first thing that comes to our mind when we have an activation function would be a threshold based classifier i.e. whether or not the neuron should be activated based on the value from the linear transformation.

In other words, if the input to the activation function is greater than a threshold, then the neuron is activated, else it is deactivated, i.e. its output is not considered for the next hidden layer. Let us look at it mathematically-

$$f(x) = \begin{cases} 1, & \text{if } x \geq 0 \\ 0, & \text{if } x < 0 \end{cases}$$



is a simple activation function used in binary classification tasks which can be created by a single if-else function.

```
def binary_step(x):
  if x < 0:
    return 0
  else:
    return 1

print(binary_step(5), binary_step(-1))
```

However, it has limitations:

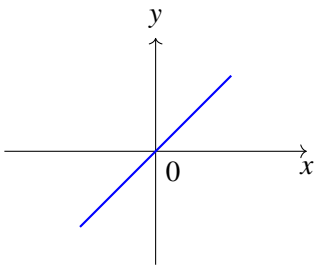
- It can only be effectively used in binary classification problems where there are only two classes in the target variable.
- The gradient of the step function is zero, hindering the backpropagation process. The derivative of $f(x)$ with respect to x is:

$$f'(x) = 0, \text{ for all } x$$

This results in the weights and biases not updating during backpropagation.

2.3.2 Linear Function

The linear function, commonly expressed as $f(x) = x$, resembles a straight line. Regardless of the number of layers, if all are linear, the final activation function of the last layer remains a linear function of the input from the first layer.



Equation: The equation for a linear function is

$$y = x$$

.

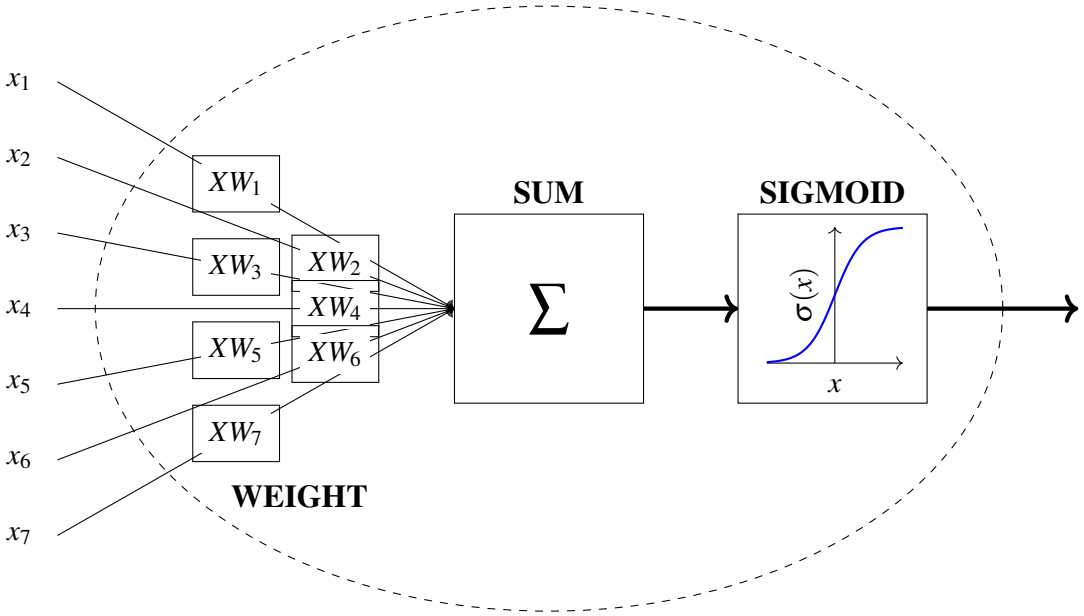
Range: The range of a linear function spans from negative infinity to positive infinity.
Uses: Linear activation functions are primarily employed at the output layer.

Issues: Differentiating the linear function to introduce non-linearity results in a constant function, as it no longer depends on the input x. This lack of non-linearity may limit the model's ability to capture complex patterns.

Example: When predicting the price of a house, a linear activation function can be suitable for the output layer since house prices can vary widely. However, non-linear functions are essential in the hidden layers to introduce complexity and enable the neural network to learn intricate relationships.

2.3.3 Sigmoid Function

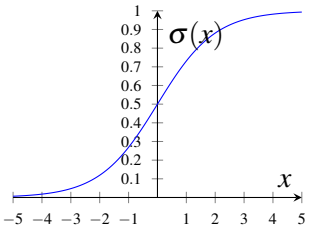
The sigmoid function is one of the most used activation functions in machine learning and deep learning. It can be used in the hidden layers, which take the previous layer's output and bring the input values between 0 and 1. Now while working with neural networks, it is necessary to calcu-late the derivate of the activation function.



The formula of the sigmoid activation function is:

$$F(x) = \frac{1}{1+e^{-x}}$$

The sigmoid function, often referred to as the squashing function, exhibits an S-shaped curve. It maintains continuity and differentiability across its entire domain. This function compresses input values from the preceding hidden layer into a range between 0 and 1. Consequently, regardless of the magnitude of the input, the sigmoid function always outputs a value within the [0, 1] interval. Graphically, the sigmoid function looks like this,



Derivative of the Sigmoid Function
In neural networks, weights and biases are initially assigned randomly and updated during backpropagation. During this process, derivatives of both the activation function and the sigmoid function are calculated. Unlike other activation functions, the sigmoid function has its own derivative function.

Let's try to derive the same. The formula of the Sigmoid Function:

$$F(x) = \frac{1}{1+e^{-x}}$$

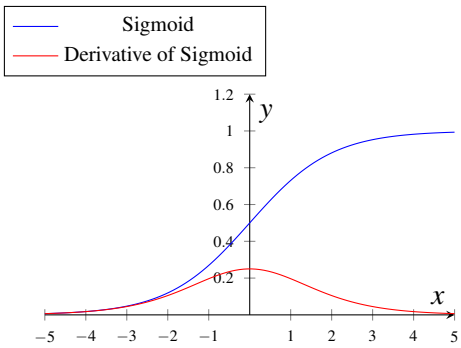
Step 1: Derivation concerning x on both sides.

$$F'(x) = \frac{d}{dx}\left(\frac{1}{1+e^{-x}}\right)$$

Step 2: Applying the reciprocating rule.

$$\begin{aligned} F'(x) &= -\frac{1}{(1+e^{-x})^2}\frac{d}{dx}(1+e^{-x}) \\ &= -\frac{1}{(1+e^{-x})^2}\frac{d}{dx}(-x) \\ &= -\frac{e^{-x}}{(1+e^{-x})^2}\cdot e^{-x}\frac{d}{dx}(-x) = \frac{e^{-x}}{(1+e^{-x})^2} \end{aligned}$$

The above equation is known as the **derivative of the sigmoid function**.



Step 3: Modifying the equation for a more generalized form.

$$\begin{aligned} \sigma'(x) &= \frac{e^{-x}}{(1+e^{-x})^2} \\ &= \frac{1+e^{-x}-1}{(1+e^{-x})^2} \\ &= \frac{1}{1+e^{-x}} - \frac{1}{(1+e^{-x})^2} \\ &= \sigma(x)\cdot\left(1-\frac{1}{1+e^{-x}}\right) = \sigma(x)(1-\sigma(x)) \end{aligned}$$

The above equation is known as the **generalized form of the sigmoid function**.

Code to Implement Sigmund Function

One must write the following code to implement the sigmoid function in Python. The below required the value of x to be pre-defined to get the sigmoid deal out of it. Suppose the input weights are fed to the layers, and the consequences and biases are passed into the next layer. If the final output layer has a sigmoid function, it will be applied to the output, and the final result will be displayed.
For example, Let's suppose the output from the hidden layer is 1; then, the value of x would be 1.
Final Output :

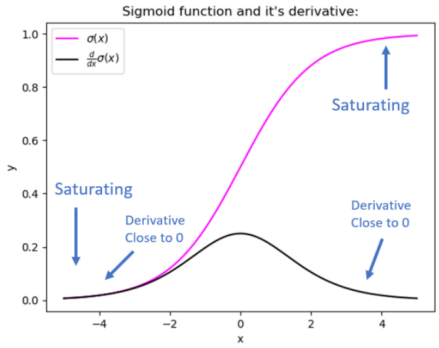
$$\begin{aligned}\sigma(x) &= \frac{1}{1+e^{-x}} \\ &= \frac{1}{1+e^{-1}} \\ &= \frac{1}{1+0.367} \\ &= \frac{1}{1.367} \\ &\approx 0.7315\end{aligned}$$

As we can see here, the output from the previously hidden layer was 1, and the function made it 0.7315, where it is visible that the Sigmoid Function is a Squeezing Function.

Applications of Sigmoid Function

1. **Binary Classification Problems :** We can use the sigmoid function in binary classification problems as it returns the output between 0 and 1.
2. **Probabilistic Models :** We can use the sigmoid function when we are required to work on a probabilistic model as it can be used to calculate the probability of a given class between 0 and 1.
3. **Image Datasets and Neural Networks :** The sigmoid function can be used for neural networks on image datasets for performing tasks like image segmentations, classifications, etc.

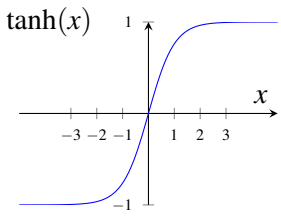
Limitations of Sigmoid Function



1. **Vanishing Gradient Problem:** One of the significant issues with the sigmoid is the lack of weight updating. Therefore, the function sometimes returns small values as outputs, making no changes in the weights and biases that cause the vanishing gradient problem.
2. **Exploding Gradient Problem:** It also sometimes happens that the sigmoid returns very large values as output, resulting in an exploding gradient problem.
3. **A Squeezing Function:** It is sometimes noted that as the sigmoid is a squeezing function, it makes the output results between 0 and 1, thereby hiding the essence or the knowledge between much higher and much smaller numbers and making the model less accurate.

2.3.4 Tanh Function

The Tanh function, short for **Hyperbolic Tangent function**, is frequently preferred over the sigmoid function due to its improved performance in neural networks. Mathematically, it's a shifted version of the sigmoid function, exhibiting a similar S-shaped curve but centered at 0 and ranging between -1 and 1. This zero-centered nature makes it beneficial for learning representations of data with both positive and negative values. Additionally, Tanh and sigmoid functions are closely related and can be derived from each other through simple mathematical transformations. Overall, the Tanh function is a popular choice in neural network architectures, offering advantages in training stability and representation learning compared to the sigmoid function.



Equation : The equation for a linear function is

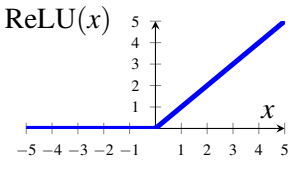
$$F(x) = \frac{2}{1+e^{-2x}} - 1$$

Range : The range of a Tanh function spans from -1 to 1.
Nature : non-linear
Uses : Usually used in hidden layers of a neural network as it's values lies between -1 to 1 hence the mean for the hidden layer comes out be 0 or very close to it, hence helps in centering the data by bringing mean close to 0. This makes learning for the next layer much easier.

Issues: Tanh encounters vanishing gradient issues, saturation at extremes, and computational complexity.

2.3.5 RELU Function

ReLU, short for Rectified Linear Unit, stands as one of the most prevalent activation functions in modern neural networks. Primarily deployed within the hidden layers of these networks, ReLU operates by outputting the input directly if it is positive, and zero otherwise. Its widespread adoption stems from its simplicity and computational efficiency, which contribute to faster training times and improved performance on a variety of tasks. **RELU learns much faster than sigmoid and Tanh function**



Equation :

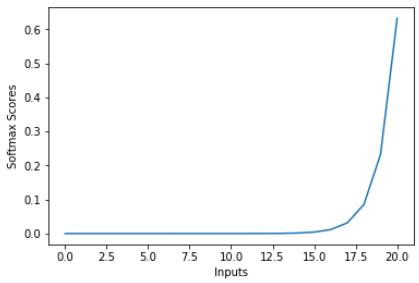
$$A(x) = max(0,x)$$

It gives an output x if x is positive and 0 otherwise.
Range : The range of a RELU function spans from 0 to $+\infty$.
Nature : non-linear, which means we can easily backpropagate the errors and have multiple layers of neurons being activated by the ReLU function.
Uses : ReLu is less computationally expensive than tanh and sigmoid because it involves simpler mathematical operations. At a time only a few neurons are activated making the network sparse making it efficient and easy for computation.

Issues: ReLU suffers from the dying ReLU problem, unbounded activation for positive inputs, and lack of differentiability at zero.

2.3.6 Softmax Function

Softmax, a variant of the sigmoid function, is particularly useful for multi-class classification tasks, while the ReLU function serves as a versatile choice for hidden layers in neural networks. For binary classification outputs, the sigmoid function is a natural choice, while Softmax excels in predicting class probabilities for multi-class scenarios.

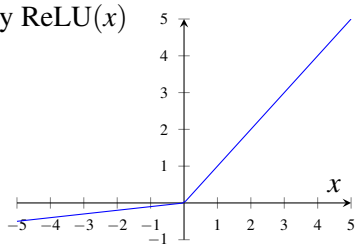


Nature : non-linear
Uses : Usually used when trying to handle multiple classes. the softmax function was commonly found in the output layer of image classification problems.The softmax function would squeeze the outputs for each class between 0 and 1 and would also divide by the sum of the outputs.
Output : The softmax function is ideally used in the output layer of the classifier where we are actually trying to attain the probabilities to define the class of each input.

2.3.7 Leaky ReLU Function

Leaky ReLU, an extension of the ReLU function, mitigates the **"dying ReLU"** issue by introducing a small gradient for negative inputs. This adjustment prevents neurons from becoming inactive during training, enhancing the model's resilience.

Leaky ReLU(x)



Equation :

$$F(x) = \begin{cases} x & \text{if } x > 0, \\ \alpha x & \text{otherwise,} \end{cases}$$

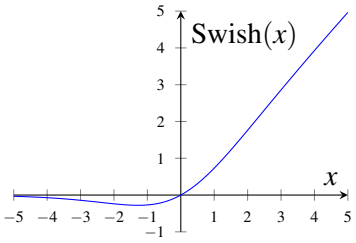
Derivative :

$$F'(x) = \begin{cases} 1 & \text{if } x > 0, \\ \alpha & \text{otherwise,} \end{cases}$$

Range : $[\alpha x, +\infty)$, where α is a small positive constant (e.g., 0.01)
Nature : non-linear, which means we can easily backpropagate the errors and have multiple layers of neurons being activated by the ReLU function.
Uses : Relu return 0 if the input is negative and hence the neuron becomes inactive as it does not contribute to gradient flow. Leaky Relu overcomes this problem by allowing small value to flow when the input is negative. So, if the learning is too slow using Relu, one can try using Leaky Relu to see any improvement happens or not.

2.3.8 Swish Function

In 2017, the Google Brain team introduced the Swish activation function as a promising alternative to ReLU. While ReLU has been pivotal in reviving interest in deep learning during the second AI winter, Swish has emerged as a compelling option for deeper networks, showing superior performance in various experiments. Although some sources refer to Swish as the sigmoid-weighted linear unit or SiLU, these terms are less common. The function is expressed as $x \times \sigma(x)$, leveraging the sigmoid function's characteristics. While sigmoid was historically significant, it has become less prevalent due to the vanishing gradient problem. However, a slight modification has revitalized its importance in modern deep learning frameworks.



Equation :

$$F(x) = \frac{x}{1+e^{-x}}$$

Range : Bounded below but Unbounded above: Y approach to constant value at X approaches negative infinity but Y approach to infinity as X approaches ∞.

Derivative of the function will be involved in the backpropagation step.

$$y = x \cdot \sigma(x) = x \cdot \left(\frac{1}{1+e^{-x}} \right)$$

The equation consists of two differentiable functions. By applying the product rule we get.

$$y' = x' \cdot \sigma(x) + x \cdot \sigma(x)'$$

Remember the derivative of the sigmoid function.

$$\sigma(x) = \frac{1}{1+e^{-x}}$$

$$\sigma(x)' = \sigma(x) \cdot (1 - \sigma(x))$$

Derivative of the function includes the derivative of the sigmoid, too.

$$y' = x' \cdot \sigma(x) + x \cdot \sigma(x)'$$

$$y' = \sigma(x) + x \cdot \sigma(x) \cdot (1 - \sigma(x)) = \sigma(x) + x \cdot \sigma(x) - x \cdot \sigma^2(x)$$

The second term is equal to the identity of the Swish function. Shift it to the left.

$$y' = x \cdot \sigma(x) + \sigma(x) - x \cdot \sigma^2(x) = y + \sigma(x) - x \cdot \sigma^2(x)$$

Now, the 2nd and 3rd terms both have a sigmoid multiplier. Let's express them both as a common sigmoid parenthesis.

$$y' = y + \sigma(x) \cdot (1 - x \cdot \sigma(x))$$

The term in the parenthesis includes the Swish function again. We can express it as the function y.

$$y' = y + \sigma(x) \cdot (1 - y)$$

This is the most basic form for the derivative of the Swish function.

2.3.9 Choosing Activation Functions for Neural Networks

The choice of activation function depends on the problem type and the expected output range. For instance, ReLU is suitable for predicting values larger than 1, while sigmoid or tanh are better for output ranges of (0,1) or (-1, 1). In classification tasks where predicting a probability distribution over mutually exclusive class labels is necessary, softmax should be used in the last layer. However, ReLU is commonly recommended for hidden layers due to its effectiveness.

For binary classification, the Sigmoid activation function is preferred, while tanh and sigmoid perform poorly for hidden layers. ReLU or its variant, leaky ReLU, are recommended for hidden layers. Softmax is ideal for multiclass classification tasks. While there are other activation functions available, these are among the most commonly used ones.

Activation Functions and their Derivatives

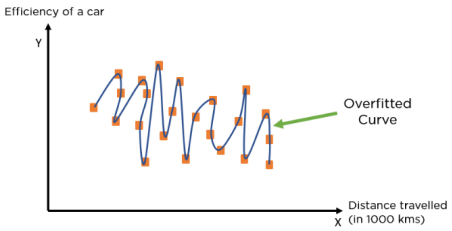
Function Type	Equation	Derivative
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma'(x) = \sigma(x) \cdot (1 - \sigma(x))$
Tanh	$\tanh(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}$	$\tanh'(x) = 1 - \tanh^2(x)$
ReLU	$\text{ReLU}(x) = \max(0, x)$	$\text{ReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases}$
Parametric ReLU	$\text{PReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha x & \text{otherwise} \end{cases}$	$\text{PReLU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha & \text{otherwise} \end{cases}$
Swish	$\text{Swish}(x) = \frac{x}{1+e^{-x}}$	$\text{Swish}'(x) = \text{Swish}(x) + \frac{1}{1+e^{-x}}(1 - \text{Swish}(x))$
Softmax	$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$	$\text{Softmax}'(x) = \text{Softmax}(x) \cdot (1 - \text{Softmax}(x))$
ELU	$\text{ELU}(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^{x} - 1) & \text{otherwise} \end{cases}$	$\text{ELU}'(x) = \begin{cases} 1 & \text{if } x > 0 \\ \alpha e^{x} & \text{otherwise} \end{cases}$

2.4 Capacity of Model

The capacity of a deep learning neural network model determines its ability to learn various mapping functions. It is governed by the configuration of nodes and layers. Insufficient capacity leads to underfitting, where the model fails to grasp the training dataset. Conversely, excessive capacity can result in overfitting, causing the model to memorize the training data or encounter optimization challenges. Finding the right balance is crucial for effective learning and generalization.

2.4.1 Overfitting

Overfitting occurs when a model performs exceptionally well on training data but struggles with new, unseen test data. This phenomenon arises when the model learns intricate details and noise from the training data, adversely affecting its performance on test data. Overfitting is often attributed to low bias and high variance. Bias error stems from incorrect assumptions in the learning algorithm, leading to underfitting, where the model overlooks crucial relationships between features and target outputs.



Reasons for Overfitting:

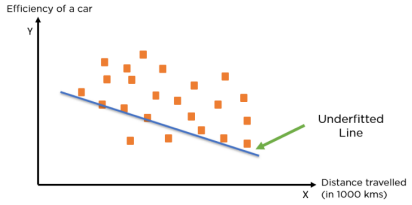
- Data used for training is not cleaned and contains noise (garbage values) in it.
- The model has a high variance.
- The size of the training dataset used is not enough.
- The model is too complex.

Ways to Tackle Overfitting:

- Using K-fold cross-validation.
- Using Regularization techniques such as Lasso and Ridge.
- Training model with sufficient data.
- Adopting ensembling techniques.

2.4.2 Underfitting

When a model has not learned the patterns in the training data well and is unable to generalize well on the new data, it is known as underfitting. An underfit model has poor performance on the training data and will result in unreliable predictions. Underfitting occurs due to high bias and low variance. The variance is an error from sensitivity to small fluctuations in the training set. High variance may result in modeling the random noise in the training data. This is called overfitting.

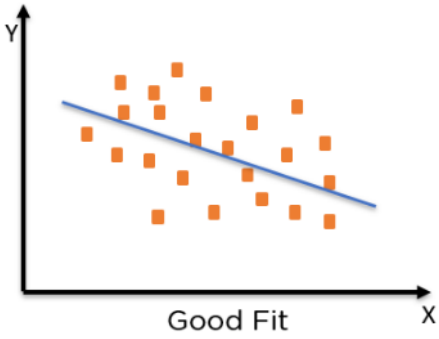


Reasons for Underfitting:

- Data used for training is not cleaned and contains noise (garbage values) in it.
- The model has a high bias.
- The size of the training dataset used is not enough.
- The model is too simple.

Ways to Tackle Underfitting:

- Increase the number of features in the dataset.
- Increase model complexity.
- Reduce noise in the data.
- Increase the duration of training the data.



Understanding overfitting and underfitting is crucial in machine learning. A well-fitted model is determined by observing its performance over time with training data. As the algorithm learns, the error on the training data decreases, along with the error on the test dataset. However, training a model for an extended period may cause it to learn unnecessary details and noise, resulting in overfitting. Achieving a good fit involves stopping the training process when the error begins to increase, ensuring a balanced and effective model.

2.5 Hyperparameter

A hyperparameter is a configuration setting for a machine learning model that must be chosen before the training process begins. Unlike parameters, which are learned during training, hyperparameters remain fixed and influence how the model learns and performs.

2.5.1 Hyperparameters in Simple Neural Networks

Let's explore some key hyperparameters in neural networks, how they can be tuned, and their effects:

1. **Learning Rate**:
   - Determines the speed at which the model updates its parameters during training.
   - Too small a learning rate leads to slow convergence, while too large a rate may cause overshooting.
   - A decaying learning rate, starting high and decreasing over time, is often beneficial to balance speed and accuracy.
2. **Number of Iterations for Training**:
   - Defines how many times the model goes through forward and backward propagation.
   - Early stopping, halting training when the loss starts to increase, can prevent overfitting but may complicate optimization.
3. **Number of Hidden Layers**:
   - Refers to layers between input and output layers.
   - Too few layers can result in underfitting, while too many may lead to overfitting.
   - A moderate number of layers with regularization is often recommended to prevent overfitting.
4. **Number of Neurons in Each Layer**:
   - Determines the number of nodes in hidden layers.
   - Insufficient neurons may cause underfitting, while excessive neurons may cause overfitting.
   - Balanced neuron counts or gradually increasing/decreasing counts across layers are common strategies.
5. **Activation Functions**:
   - Non-linear functions applied to each layer's output.
   - Enable the network to learn complex patterns in the data.
   - Common choices include Rectified Linear Unit (ReLU) for hidden layers and suitable functions for output layers based on the task.
6. **Weight Initialization**:
   - The method used to set initial weights, impacting learning speed and convergence.

- Proper initialization helps prevent issues like vanishing or exploding gradients.
- Random numbers from a Gaussian distribution with a small standard deviation are often used.

7. **Regularization Type and Parameters**:
   - Techniques to prevent overfitting by adding constraints to the model.
   - Popular methods include L1 and L2 regularization, and Dropout.
   - Each regularization type has parameters that control its impact on the model's complexity.

2.6 Data Splitting: Training, Testing, and Validation Datasets

In machine learning, dividing data into training, testing, and validation datasets is essential for model evaluation and performance assessment.

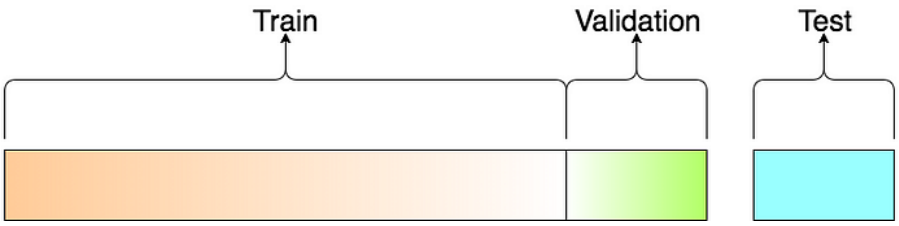


Figure 2.1: Data Splitting

2.6.1 Training Dataset

This is the sample of data used to fit the model. It's the actual dataset used to train the model, where it learns from the data by adjusting its weights and biases (in the case of a Neural Network).

2.6.2 Validation Dataset

- The validation dataset is used to provide an unbiased evaluation of a model fit on the training dataset while tuning model hyperparameters.
- Machine learning engineers use this dataset to fine-tune the model hyperparameters. The model occasionally sees this data, but it doesn't directly learn from it.
- The validation set is also known as the Dev set or Development set, as it aids in the development stage of the model.

2.6.3 Test Dataset

- The test dataset is used to provide an unbiased evaluation of a final model fit on the training dataset.
- It serves as the gold standard for evaluating the model's performance.
- The test dataset is only used once the model is completely trained using the training and validation sets.
- It's generally well-curated and contains carefully sampled data representing various classes the model would face in real-world scenarios.
- Using the validation set as the test set is not a good practice, as the test set is specifically curated for evaluating the final model's performance.

By properly splitting data into these three sets, we can accurately evaluate our models and ensure they perform well in real-world applications.

2.7 Estimator

In deep learning, an estimator is an equation for picking the “best,” or most likely accurate, data model based upon observations in reality. Not to be confused with estimation in general, the estimator is the formula that evaluates a given quantity (the estimand) and generates an estimate. This estimate is then inserted into the deep learning classifier system to determine what action to take.

2.7.1 Uses of Estimators

Estimators play a fundamental role in translating the theoretical concepts of deep learning into practical implementation. They enable the process of parameter estimation within datasets, crucial for tasks like configuring the layers in a neural network or determining the bandwidth in a kernel. Without effective estimators, an AI system would lack the ability to "learn."

A classic illustration of estimators in action is the "German Tank Problem" during World War Two. Faced with uncertainty about the number of tanks being produced by the Germans, Allied statisticians devised a strategy based on estimating the total count using the serial numbers of captured or destroyed tanks. By applying a rule-based estimator equation to these serial numbers and employing minimum variance analysis, they could generate the most probable estimate for the ongoing production of German tanks.

2.7.2 Classification of Estimators

Estimators are typically categorized into two main types: point and interval. Point estimators yield singular values, such as mean or standard deviation, which can be directly utilized in classifier functions within deep learning algorithms. On the other hand, interval estimators provide a range of plausible values, such as confidence intervals, for further analysis.

Moreover, various characteristics define the behavior of estimator rules, leading to different types of estimates:

- **Biased:** Produces estimates that tend to either overestimate or underestimate the true value.
- **Efficient:** Generates estimates with the smallest possible variance, considered the most optimal.
- **Invariant:** Offers rigid estimates that remain unchanged despite transformations in the dataset.
- **Shrinkage:** Combines raw estimates with other variables to produce more sophisticated estimations.
- **Sufficient:** Derives estimates for the entire population parameter from a limited sample dataset.
- **Unbiased:** Provides an estimate that exactly matches the true value without any systematic overestimation or underestimation.

2.7.3 Bias and Variance

The **bias** is an error from wrong assumptions in the learning algorithm. High bias can cause an algorithm to miss the relevant relations between features and target outputs. This is called underfitting.

The **variance** is an error from sensitivity to small fluctuations in the training set. High variance may result in modeling the random noise in the training data. This is called overfitting.

The **bias-variance** tradeoff is a term to describe the fact that we can reduce the variance by increasing the bias. Good regularization techniques strive to simultaneously minimize the two sources of error. Bias is the measurement of deviation or error from the real value of function; variance is the measurement of deviation in the response variable function while estimating it over a different training sample of the dataset.

Therefore, for a generalized data model, we must keep bias possibly low while modeling that leads to high accuracy. And, one should not obtain greatly varied results from the output, therefore, low variance is recommended for a model to perform good.

2.8 Deep Networks

A deep network is a neural network with some level of complexity, usually at least two layers, qualifies as a deep neural network (DNN), or deep net for short. Deep nets process data in complex ways by employing sophisticated math modeling. A deep neural network (DNN) is a type of neural network with multiple layers between the input and output layers. These layers allow the network to model and process data in complex ways, using advanced mathematical techniques to learn patterns and make predictions.

2.8.1 Feed forward networks

Feed forward neural networks are artificial neural networks in which nodes do not form loops. This type of neural network is also known as a multi-layer neural network as all information is only passed forward.

During data flow, input nodes receive data, which travel through hidden layers, and exit output nodes. No links exist in the network that could get used to by sending information back from the output node.

2.9 Working Principle of a Feed Forward Neural Network

A simplified version of a feed-forward neural network is the single-layer perceptron, often used for classification tasks. Here's how it works:

2.9.1 Single-Layer Perceptron

- Inputs are multiplied by corresponding weights as they enter the layer.
- These weighted inputs are then summed together.
- If the sum exceeds a predefined threshold (usually zero), the perceptron outputs 1; otherwise, it outputs -1.
- This binary output mechanism makes it effective for simple classification tasks.

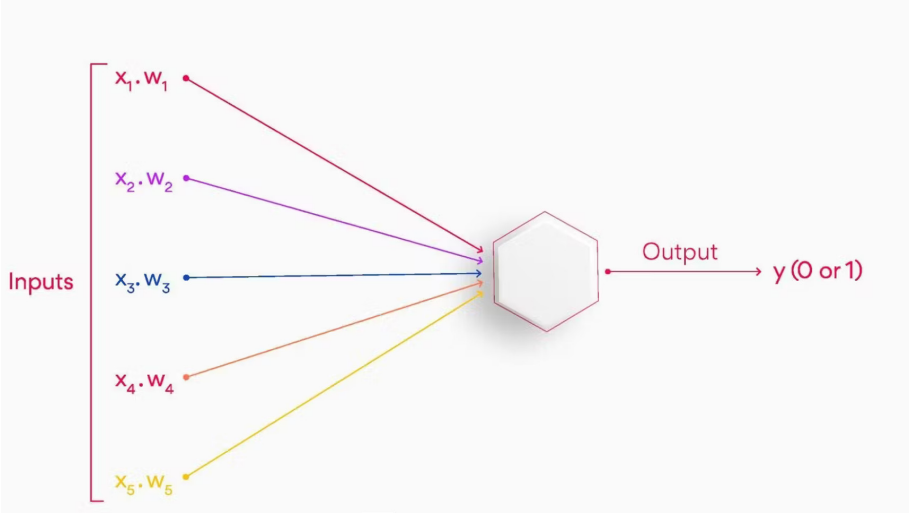


Figure 2.2: Single-Layer Perceptron

Training

- Machine learning enhances single-layer perceptrons through training.
- During training, the perceptron adjusts its weights based on the difference between actual and desired outputs, using the delta rule.
- This iterative weight adjustment process minimizes prediction errors and is known as gradient descent.

Multi-Layer Perceptrons (MLPs)

- These networks, more complex than single-layer perceptrons, use back-propagation for training.
- Back-propagation involves propagating errors backward from the output layer to the hidden layers.
- Each layer's weights are adjusted based on the error contribution from the final output.
- This enables fine-tuning of weights across multiple layers, leading to more accurate models.

2.10 Layers in a Feed Forward Neural Network

2.10.1 Input Layer

- Receives input and passes it on to other layers.
- The number of neurons in this layer should match the number of features in the dataset.

2.10.2 Output Layer

- The neurons of this layer receive input and pass it on to the other layers of the network.
- Feature or attribute numbers in the dataset must match the number of neurons in the input layer.

2.10.3 Hidden Layer

- Separates input and output layers.
- Can have multiple hidden layers, each consisting of several neurons.
- Neurons in hidden layers transform input before passing it to the next layer.
- Network gets constantly updated with weights in order to make it easier to predict.

2.11 Key Components

2.11.1 Neuron Weights

Neurons are connected by a weight, which measures their strength or magnitude. Similar to linear regression coefficients, input weights can also be compared.

Weights are normally between 0 and 1, with a value between 0 and 1.

2.11.2 Neurons

Artificial neurons are used in feed-forward networks, which are adapted from biological neurons. A neural network consists of artificial neurons.

Neurons function in two ways: first, they create weighted input sums, and second, they activate the sums to normalize them.

Activation functions can either be linear or nonlinear. Neurons have weights based on their inputs. During the learning phase, the network studies these weights.

2.11.3 Activation Function

Neurons are responsible for making decisions in this area.

According to the activation function, the neurons determine whether to make a linear or nonlinear decision. Since it passes through so many layers, it prevents the cascading effect from increasing neuron outputs.

2.11.4 Function in feed forward neural network

Cost function

Cost function measures the performance of a machine learning model for given data. Cost function quantifies the error between predicted and expected values and present that error in the form of a single real number. Depending on the problem, cost function can be formed in many different ways.

$$C(w,b) \equiv \frac{1}{2n}\sum_{x} \|y(x)-a\|^2$$

where,

$$\begin{aligned} w &= \text{the weights gathered in the network} \\ b &= \text{biases} \\ n &= \text{number of inputs for training} \\ a &= \text{output vectors} \\ x &= \text{input} \\ \|v\| &= \text{vector } v\text{'s normal length} \end{aligned}$$

Loss function

The loss function of a neural network is used to determine if an adjustment needs to be made in the learning process.

Neurons in the output layer are equal to the number of classes, showing the differences between predicted and actual probability distributions. The following is the cross-entropy loss for binary classification:

$$\text{Cross Entropy Loss:} \quad L(\Theta) = \begin{cases} -\log(\hat{y}) & \text{if } y = 1 \\ -\log(1-\hat{y}) & \text{if } y = 0 \end{cases}$$

2.11.5 Learning XOR

The XOR, or "exclusive OR", problem is a classic problem in the field of artificial intelligence and machine learning. It is a problem that cannot be solved by a single layer perceptron, and therefore requires a multi-layer perceptron or a deep learning model.

The XOR function

The XOR function is a binary function that takes two binary inputs and returns a binary output. The output is true if the number of true inputs is odd, and false otherwise. In other words, it returns true if exactly one of the inputs is true, and false otherwise.

2.12 The XOR Problem Solution

2.12.1 Introduction to XOR Problem

The XOR (exclusive OR) problem is a classic example in the study of neural networks. It is defined by the following truth table:

Table 2.1: Truth Table for XOR

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

A single-layer perceptron can only solve linearly separable problems, meaning that it can only separate inputs with a single straight line. However, the XOR problem is not linearly separable, as there is no single straight line that can separate the classes 0 and 1 in the XOR truth table.

2.12.2 Limitation of Single-Layer Perceptron

To understand why a single-layer perceptron cannot solve the XOR problem, consider the XOR function plotted on a coordinate plane:

- Points (0,0) and (1,1) belong to class 0.
- Points (0,1) and (1,0) belong to class 1.

2.12.3 Graphical Representation of XOR Problem

The following graph shows the XOR problem plotted on a coordinate plane. The points (0,0) and (1,1) belong to class 0, while the points (0,1) and (1,0) belong to class 1.

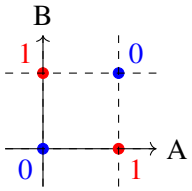


Figure 2.3: XOR Problem on a Coordinate Plane

2.12.4 Limitation of Single-Layer Perceptron

A single-layer perceptron can only solve linearly separable problems, meaning it can only separate inputs with a single straight line. In the graph above, there is no single straight line that can separate the blue points (class 0) from the red points (class 1).

For example, if we try to draw a line to separate the points, we will see that it is impossible to place a straight line that will correctly classify all points:

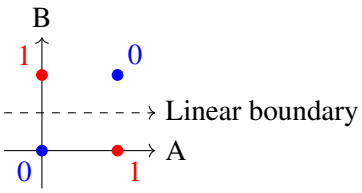


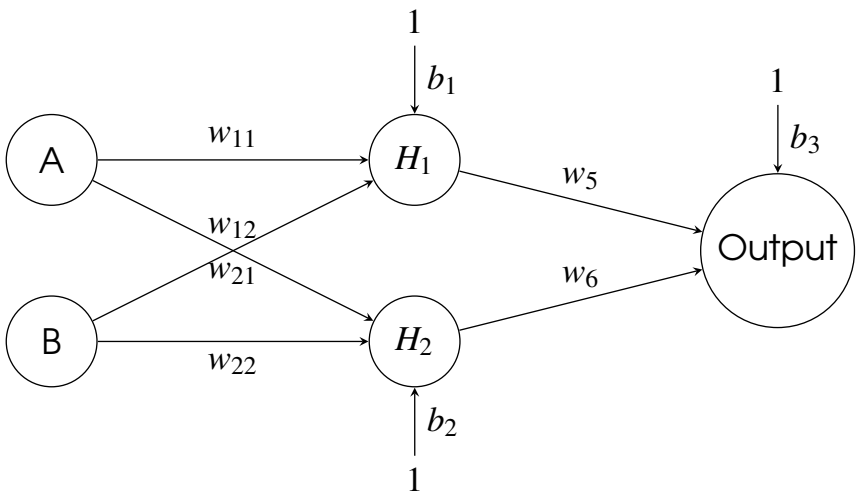
Figure 2.4: Incorrect Linear Boundary for XOR Problem

As seen in the diagram, any attempt to draw a single linear boundary will always misclassify at least one of the points. This demonstrates that a single-layer perceptron cannot solve the XOR problem because it cannot create the necessary non-linear decision boundary.

There is no single line that can separate these points into their respective classes, thus making it impossible for a single-layer perceptron to classify them correctly.

2.12.5 Solution Using Multilayer Perceptron

A multilayer perceptron (MLP) can solve the XOR problem by introducing hidden layers that enable the network to learn more complex functions. Here is a diagram of an MLP with one hidden layer that can solve the XOR problem:



The multilayer perceptron shown in the diagram has two input neurons, two hidden neurons, and one output neuron. The hidden neurons enable the network to create a non-linear decision boundary, which can successfully classify the XOR problem.

To solve the XOR problem, the weights and biases can be set as follows (these values are just one of the possible sets that can work):

- Weights from inputs to hidden layer:

$$w_{11} = 1, \qquad w_{12} = 1$$
$$w_{21} = 1, \qquad w_{22} = 1$$

- Biases for hidden layer neurons:

$$b_1 = -1.5, \qquad b_2 = -0.5$$

- Weights from hidden layer to output neuron:

$$w_{31} = 1, \qquad w_{32} = -2$$

- Bias for output neuron:

$$b_3 = -0.5$$

The network works as follows:

- The input layer receives the inputs A and B.
- The hidden layer processes these inputs using the weights and biases to generate non-linear transformations:

$$H1 = \sigma(A \cdot w_{11} + B \cdot w_{21} + b_1)$$
$$H2 = \sigma(A \cdot w_{12} + B \cdot w_{22} + b_2)$$

where σ is the activation function (e.g., sigmoid).

- The output layer combines the transformed inputs from the hidden layer:

$$\text{Output} = \sigma(H1 \cdot w_{31} + H2 \cdot w_{32} + b_3)$$

- The network produces the correct XOR output by adjusting the weights and biases during training to minimize the error.

By introducing hidden layers, the MLP overcomes the limitation of the single-layer perceptron and can solve the XOR problem by learning the complex relationships between the inputs and the outputs.

2.13 Gradient-Based Learning

Gradient-based learning is a method to optimize the weights and biases of a neural network by minimizing a loss function. This is typically done using the gradient descent algorithm.

2.13.1 Gradient Descent Algorithm

Gradient-based learning is the backbone of many deep learning algorithms. This approach involves iteratively adjusting model parameters to minimize the loss function, which measures the difference between the actual and predicted outputs. The update rule for weights w and biases b is given by:

$$w := w - \eta \frac{\partial L}{\partial w} \tag{2.1}$$

$$b := b - \eta \frac{\partial L}{\partial b} \tag{2.2}$$

where η is the learning rate, L is the loss function, and $\frac{\partial L}{\partial w}$, $\frac{\partial L}{\partial b}$ are the gradients of the loss function with respect to w and b, respectively.

2.13.2 Loss Function

A common loss function used in neural networks is the mean squared error (MSE), defined as:

$$L(w,b) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

where y_i are the actual values, $\hat{y}_i$ are the predicted values, and n is the number of training examples.

2.13.3 Gradient Computation

The gradients of the loss function with respect to the weights and biases are computed as follows:

$$\frac{\partial L}{\partial w} = -\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i) x_i \tag{2.3}$$

$$\frac{\partial L}{\partial b} = -\frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i) \tag{2.4}$$

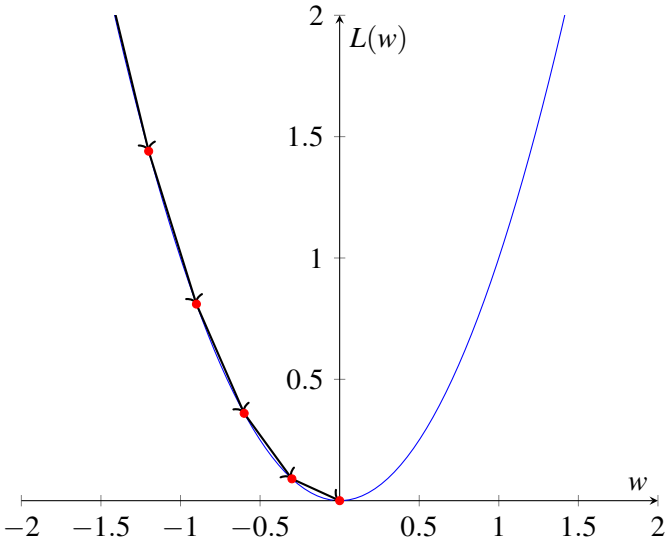


Figure 2.5: Illustration of Gradient Descent Process

2.14 Hidden Units – Architecture Design

2.14.1 Definition and Role of Hidden Units

Hidden units, also known as hidden neurons, are neurons situated between the input and output layers of a neural network. They process inputs received from the input layer, apply a weighted sum, add a bias, and pass the result through an activation function to introduce non-linearity. This transformed information is then passed to the output layer or the next hidden layer.

2.14.2 Weight Updation in Hidden Units

The weights of the connections between neurons are updated using the backpropagation algorithm, which involves:

1. **Forward Pass**: Calculate the output of the network by passing the inputs through the hidden units to the output layer.
2. **Loss Calculation**: Compute the loss function, which measures the difference between the predicted output and the actual target.
3. **Backward Pass**: Use the chain rule to compute the gradients of the loss function with respect to each weight.
4. **Weight Update**: Adjust the weights using the gradient descent algorithm.

The weight update rule for a weight w_{ij} connecting neuron i to neuron j is given by:

$$w_{ij} := w_{ij} - \eta \frac{\partial L}{\partial w_{ij}}$$

where η is the learning rate and $\frac{\partial L}{\partial w_{ij}}$ is the gradient of the loss function with respect to w_{ij}.

2.14.3 Diagram: Neural Network Architecture

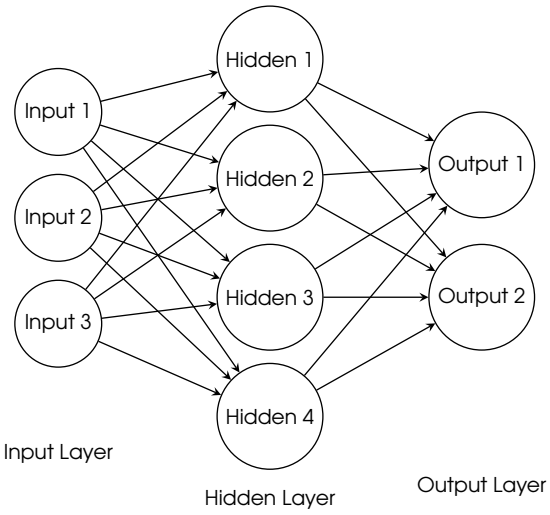


Figure 2.6: Neural Network Architecture with One Hidden Layer

2.14.4 Equations and Activation Functions

The operation of a single hidden neuron j can be described by the following equations:

$$z_j = \sum_i w_{ij} x_i + b_j$$

$$a_j = \sigma(z_j)$$

where z_j is the weighted sum of inputs x_i with weights w_{ij} and bias b_j, and σ is the activation function (e.g., ReLU, sigmoid, tanh).

2.15 Backpropagation Algorithm

2.15.1 Forward Pass

Given an input vector $\mathbf{x}$, the forward pass involves computing the activations for each layer:

$$\begin{aligned}
&\text{Input layer:} && \mathbf{x} \\
&\text{Hidden layer:} && \mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{x} + \mathbf{b}^{(1)}, \quad \mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)}) \\
&\text{Output layer:} && \mathbf{z}^{(2)} = \mathbf{W}^{(2)}\mathbf{a}^{(1)} + \mathbf{b}^{(2)}, \quad \mathbf{a}^{(2)} = \hat{\mathbf{y}} = \sigma(\mathbf{z}^{(2)})
\end{aligned}$$

where $\mathbf{W}$ are the weights, $\mathbf{b}$ are the biases, σ is the activation function, and $\hat{\mathbf{y}}$ is the predicted output.

2.15.2 Loss Calculation

The loss function L measures the difference between the predicted output $\hat{\mathbf{y}}$ and the actual target $\mathbf{y}$. For example, the Mean Squared Error (MSE) is given by:

$$L = \frac{1}{2} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2$$

2.15.3 Backward Pass

The backward pass involves computing the gradients of the loss function with respect to each weight and bias. This is done using the chain rule.

Output Layer

$$\delta^{(2)} = \frac{\partial L}{\partial \mathbf{z}^{(2)}} = (\hat{\mathbf{y}} - \mathbf{y}) \circ \sigma'(\mathbf{z}^{(2)})$$

$$\frac{\partial L}{\partial \mathbf{W}^{(2)}} = \delta^{(2)} (\mathbf{a}^{(1)})^T, \quad \frac{\partial L}{\partial \mathbf{b}^{(2)}} = \delta^{(2)}$$

Hidden Layer

$$\delta^{(1)} = (\mathbf{W}^{(2)})^T \delta^{(2)} \circ \sigma'(\mathbf{z}^{(1)})$$

$$\frac{\partial L}{\partial \mathbf{W}^{(1)}} = \delta^{(1)} (\mathbf{x})^T, \quad \frac{\partial L}{\partial \mathbf{b}^{(1)}} = \delta^{(1)}$$

2.15.4 Weight Update

The weights and biases are updated using gradient descent:

$$\mathbf{W} := \mathbf{W} - \eta \frac{\partial L}{\partial \mathbf{W}}, \quad \mathbf{b} := \mathbf{b} - \eta \frac{\partial L}{\partial \mathbf{b}}$$

where η is the learning rate.

2.15.5 Example and Diagram

Consider a simple neural network with one input layer, one hidden layer, and one output layer.

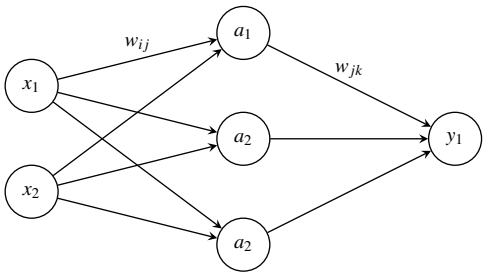


Figure 2.7: Neural Network with One Hidden Layer

For this network:

$$\mathbf{z}_1 = \mathbf{W}_{ij}\mathbf{x} + \mathbf{b}_1$$
$$\mathbf{a}_i = \sigma(\mathbf{z}_i)$$
$$\mathbf{z}_2 = \mathbf{W}_{jk}\mathbf{a}_i + \mathbf{b}_2$$
$$\hat{\mathbf{y}} = \sigma(\mathbf{z}_2)$$

Assume we have the following values for an example: - Input: $\mathbf{x} = [1, 0]$ - Weights and biases (initialized randomly)

During the forward pass, compute the activations of the hidden layer and the output. Calculate the loss using the target values. In the backward pass, compute the gradients with respect to the loss function and update the weights accordingly.

2.15.6 Back propagation

Backpropagation is a crucial concept in **neural networks**, especially for tasks like **data classification**. To achieve this, we need to update the **weights** and **biases** of the network. In **linear regression**, we use **gradient descent** to optimize parameters. Similarly, we use gradient descent in neural networks through a method called backpropagation.

Gradient Calculation

Backpropagation works by calculating the **gradient of the error function** for a single training example. This algorithm leverages the **chain rule** to efficiently train artificial neural networks using a gradient descent approach.

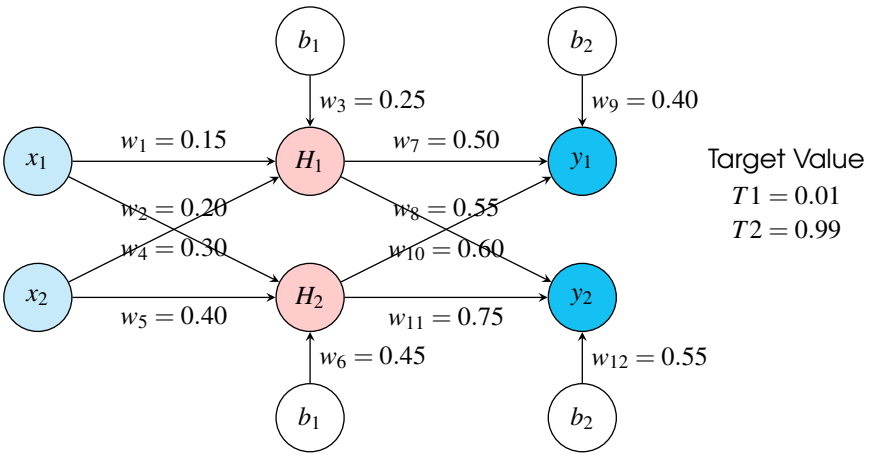
Forward and Backward Pass

The process consists of two main passes: the **forward pass** and the **backward pass**. In the forward pass, the **input data** is propagated through the network to generate an **output**. Then, in the backward pass, the **error** is propagated back from the **output layer** to the **input layer**, passing through the **hidden layers**. This reverse flow of error propagation helps update the network weights to minimize the error.

Key Aspects

The key aspects of backpropagation include its **iterative**, **recursive**, and **efficient** nature. It continuously updates the weights to improve the network's performance until it can no longer improve. For backpropagation to work, the **derivatives of the activation functions** must be known at the network design time.

In summary, backpropagation is the algorithm used to update neural network weights to reduce the error. It involves propagating the error backward through the network and using gradient descent to optimize the parameters iteratively.



Backpropagation Example

Forward Pass

$$
\begin{aligned}
z_{H1} &= w1 \cdot x1 + w2 \cdot x2 + w3 \cdot b1 \\
&= 0.15 \cdot 0.05 + 0.20 \cdot 0.10 + 0.25 \cdot 0.35 \\
&= 0.0075 + 0.02 + 0.0875 = 0.115 \\
H1 &= \text{sigmoid}(0.115) \approx 0.528 \\
z_{H2} &= w4 \cdot x1 + w5 \cdot x2 + w6 \cdot b1 \\
&= 0.30 \cdot 0.05 + 0.40 \cdot 0.10 + 0.45 \cdot 0.35 \\
&= 0.015 + 0.04 + 0.1575 = 0.2125 \\
H2 &= \text{sigmoid}(0.2125) \approx 0.553 \\
z_{y1} &= w7 \cdot H1 + w8 \cdot H2 + w9 \cdot b2 \\
&= 0.50 \cdot 0.528 + 0.55 \cdot 0.553 + 0.40 \cdot 0.60 \\
&= 0.264 + 0.303 + 0.24 = 0.807 \\
y1 &= \text{sigmoid}(0.807) \approx 0.691 \\
z_{y2} &= w10 \cdot H1 + w11 \cdot H2 + w12 \cdot b2 \\
&= 0.60 \cdot 0.528 + 0.75 \cdot 0.553 + 0.55 \cdot 0.60 \\
&= 0.3168 + 0.41475 + 0.33 = 1.06155 \\
y2 &= \text{sigmoid}(1.06155) \approx 0.743
\end{aligned}
$$

2.16 Compute Error

$$
\begin{aligned}
E &= \frac{1}{2}((T1 - y1)^2 + (T2 - y2)^2) \\
&= \frac{1}{2}((0.01 - 0.691)^2 + (0.99 - 0.743)^2) \\
&= \frac{1}{2}(0.463^2 + 0.247^2) \\
&= \frac{1}{2}(0.214369 + 0.061009) \\
&= \frac{1}{2}(0.275378) \\
&= 0.137689
\end{aligned}
$$

2.17 Backward Pass

Calculate gradients and update weights:

$$
\begin{aligned}
\frac{\partial E}{\partial w7} &= \frac{\partial E}{\partial y1} \cdot \frac{\partial y1}{\partial z_{y1}} \cdot \frac{\partial z_{y1}}{\partial w7} \\
\frac{\partial E}{\partial y1} &= y1 - T1 \\
&= 0.691 - 0.01 = 0.681
\end{aligned}
$$

$$\begin{aligned}
\frac{\partial y1}{\partial z_{y1}} &= y1(1-y1) \\
&= 0.691 \cdot (1-0.691) \\
&= 0.691 \cdot 0.309 \\
&= 0.213 \\
\frac{\partial z_{y1}}{\partial w7} &= H1 \\
&= 0.528 \\
\frac{\partial E}{\partial w7} &= 0.681 \cdot 0.213 \cdot 0.528 \\
&= 0.07651 \\
w7 &\leftarrow w7 - \eta \cdot \frac{\partial E}{\partial w7} \\
&= 0.50 - 0.1 \cdot 0.07651 \\
&= 0.49235
\end{aligned}$$

Continue for all other weights similarly.

2.17.1 Differentitation in Deep learning

Differentiation plays a crucial role in deep learning because it enables the training of neural networks using gradient descent. Gradient descent is an optimization algorithm that finds the minimum of a function by moving in the direction of the negative gradient, which points towards the steepest descent.

To understand this in the context of deep learning, let's break it down:

- **Loss Function**: The function we aim to minimize is called the loss function. It measures how well the neural network is performing on the training data. A lower loss indicates better performance.
- **Gradient Calculation**: Differentiation is used to calculate the gradient of the loss function with respect to the neural network's weights. The gradient is essentially a vector of partial derivatives that shows how the loss changes with each weight.
- **Updating Weights**: Using gradient descent, we adjust the weights in the direction of the negative gradient. This adjustment helps to reduce the loss function. This process is repeated iteratively until the loss function reaches a minimum, indicating that the neural network is trained.

Differentiation isn't just limited to training neural networks. It is also utilized in other areas of deep learning, such as:

- **Architecture Search**: Differentiation can help in discovering new and more efficient neural network architectures.
- **Hyperparameter Optimization**: It aids in optimizing hyperparameters like the learning rate and the number of training epochs, improving the network's performance.
- **Transfer Learning**: Differentiation allows for the transfer of knowledge from a pre-trained neural network to a new one, speeding up the training process and enhancing performance on new tasks.

By understanding and applying differentiation, we can effectively train and optimize neural networks, advancing their capability and efficiency in various applications.

Unit III

3 Regularization Techniques in Deep Learning 53

3.1 Regularization Strategies and Techniques
3.2 Loss Function Modification: Adding Regularization Terms
3.3 Parameter Norm Penalties
3.4 L2 Regularization
3.5 L1 Regularization
3.6 Norm Penalties as Constrained Optimization
3.7 Regularized & Under-constrained Problems
3.8 Noise Robustness
3.9 Semi-Supervised Learning
3.10 Multi-Task Learning
3.11 Early Stopping of Training
3.12 Bagging
3.13 Dropout
3.14 Adversarial Training
3.15 Parameter Tying and Sharing
3.16 Sparse Representations
3.17 Tangent Distance, Tangent Prop, and Manifold Tangent Classifier
3.18 Tangent Distance for Regularization
3.19 Tangent Propagation for Robustness
3.20 Manifold Tangent Classifier for Improved Classification

3. Regularization Techniques in Deep Learning

3.1 Regularization Strategies and Techniques

Regularization strategies are crucial for reducing test set error, often at the expense of slightly increasing training error. These techniques address overfitting, where models perform well on training data but fail to generalize to unseen data. Overfitting occurs when a model learns both the underlying patterns and the noise in the training data, leading to poor performance on new data.

The primary goal of regularization is to balance fitting the training data and generalizing to new data. By adding a penalty for model complexity, regularization discourages the model from capturing noise as significant patterns.

Various regularization techniques are available, each aiming to penalize model complexity and enhance generalization. These techniques help constrain model parameters, reduce variance, and improve test data performance. Consequently, regularization is fundamental for building robust machine learning models that perform consistently across different datasets.

3.1.1 Common Regularization Techniques

- **L1 Regularization (Lasso)**: Adds a penalty equal to the absolute value of the magnitude of coefficients. This can result in sparse models where some feature weights are reduced to zero, effectively performing feature selection.
- **L2 Regularization (Ridge)**: Adds a penalty equal to the square of the magnitude of coefficients. This discourages large weights, leading to a more evenly distributed set of feature weights.
- **Elastic Net**: Combines L1 and L2 regularization, balancing the benefits of both techniques.
- **Dropout**: Temporarily drops out units (both hidden and visible) in a neural network during training, preventing the network from becoming overly reliant on specific neurons.
- **Early Stopping**: Monitors the model's performance on a validation set and stops training when performance on the validation set starts to degrade.
- **Data Augmentation**: Expands the training dataset by creating modified versions of existing data, which helps the model generalize better by learning from more diverse examples.

3.2 Loss Function Modification: Adding Regularization Terms

In mathematical optimization and decision theory, a loss function (also known as an error function) maps events or variable values to a real number representing the "cost" associated with the event. The loss function evaluates how well an algorithm models the dataset and is a function of the machine learning algorithm's parameters.

Loss Function vs. Cost Function

- **Loss Function**: Measures the error between predicted and actual values in a machine learning model. It is used to optimize the model during training and can be specific to individual samples. Common examples include mean squared error (MSE), mean absolute error (MAE), and binary cross-entropy.
- **Cost Function**: Quantifies the overall error of the model on the entire training set. It guides the optimization process by minimizing the cost or error. The cost function is typically derived from the loss function and can include additional regularization terms.

Loss Function	Cost Function
Measures the error between predicted and actual values.	Quantifies the overall error of the model on the training set.
Used to optimize the model during training.	Guides the optimization process by minimizing the cost.
Can be specific to individual samples.	Aggregates the loss values over the entire training set.
Examples include MSE, MAE, and binary cross-entropy.	Often the average or sum of individual loss values, with possible additional terms.
Evaluates model performance.	Determines direction and magnitude of parameter updates during optimization.
Different loss functions suit different tasks or problem domains.	Derived from the loss function and can include regularization terms.

Table 3.1: Comparison between Loss Function and Cost Function

3.3 Parameter Norm Penalties

Parameter norm penalties are a fundamental regularization technique used to prevent overfitting by adding an extra term to the loss function that penalizes large weights. This helps control the complexity of the model, improves its generalization ability, and enhances numerical stability.

The technique limits the capacity of models by adding a parameter norm penalty to the objective function J. The modified objective function is given by:

$$\tilde{J}(\theta;\mathbf{X},\mathbf{y}) = J(\theta;\mathbf{X},\mathbf{y}) + \alpha\Omega(\theta)$$

Here,

$\tilde{J}$ = the regularized objective function.

J = the original objective function.

α = a hyperparameter that determines the weight of the norm penalty Ω.

$\Omega(\theta)$ = represents the norm penalty applied to the parameters θ.

Setting α to 0 means **no regularization**, while larger values of α correspond to **more regularization**.

For neural networks, we typically apply parameter norm penalties to the weights of the affine transformations, while leaving the biases unregularized. This is because:

- **Weights**: Denote the relationship between variables and require observing both variables in various conditions, thus benefiting more from regularization.

- **Biases**: Control only single variables and require less data to fit accurately, hence they can be left unregularized.

By selectively regularizing the weights and not the biases, we achieve a better balance between model complexity and generalization performance.

3.4 L2 Regularization

L2 regularization, also known as Tikhonov regularization or ridge regression when applied to linear models, is a technique used to prevent overfitting by adding a penalty term to the loss function. This penalty is the sum of the squared values of the model parameters. The main objective of L2 regularization is to keep the model parameters small, simplifying the model and making it less sensitive to noise in the training data.

3.4.1 Loss Function with L2 Regularization

In a typical machine learning model, we aim to minimize a loss function $J(\theta;\mathbf{X},\mathbf{y})$, which measures the error of the model's predictions $\hat{y}$ compared to the true values $\mathbf{y}$. The model parameters are represented by θ.

With L2 regularization, the loss function is modified to include an additional term that penalizes large weights. The regularized loss function $\tilde{J}$ is given by:

$$\tilde{J}(\theta;\mathbf{X},\mathbf{y}) = J(\theta;\mathbf{X},\mathbf{y}) + \alpha\Omega(\theta)$$

where:

- $J(\theta;\mathbf{X},\mathbf{y})$ is the original loss function.
- α is the regularization parameter that controls the strength of the regularization (a non-negative hyperparameter).
- $\Omega(\theta) = \frac{1}{2}\sum_{j=1}^{n}\theta_j^2$ is the L2 penalty term (the sum of the squares of the model parameters).

3.4.2 Interpretation of the Penalty Term

The penalty term $\Omega(\theta)$ in L2 regularization is:

$$\Omega(\theta) = \frac{1}{2}\sum_{j=1}^{n}\theta_j^2$$

This term adds the squared values of all the model parameters θ_j. The factor of $\frac{1}{2}$ is included for mathematical convenience when deriving the gradient.

3.4.3 Effect on Model Training

- **Preventing Overfitting:** By penalizing large values of θ, L2 regularization encourages the model to keep the parameters small. This results in a simpler model that is less likely to overfit the training data.
- **Smooth Optimization:** The addition of the squared terms makes the loss function strongly convex, which can improve the convergence properties of optimization algorithms.

3.4.4 Gradient Descent with L2 Regularization

During gradient descent, the update rule for each parameter θ_j is modified to include the regularization term. The gradient of the regularized loss function with respect to θ_j is:

$$\frac{\partial \tilde{J}}{\partial \theta_j} = \frac{\partial J}{\partial \theta_j} + \alpha \theta_j$$

This modifies the parameter update rule to:

$$\theta_j := \theta_j - \eta \left(\frac{\partial J}{\partial \theta_j} + \alpha \theta_j \right)$$

where η is the learning rate. This update rule shows that each parameter θ_j is not only adjusted based on the gradient of the loss function but also shrunk towards zero by the term $\alpha \theta_j$.

3.4.5 Choosing the Regularization Parameter α

- **Cross-Validation:** The value of α is typically chosen using cross-validation. It controls the trade-off between fitting the training data well and keeping the model parameters small.
- **Impact:** A larger α results in stronger regularization, leading to smaller parameter values and potentially higher bias but lower variance. A smaller α results in weaker regularization, allowing the model to fit the training data more closely but with a higher risk of overfitting.

3.4.6 Example

Consider a linear regression model without regularization:

$$J(\theta; \mathbf{X}, \mathbf{y}) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

With L2 regularization, the loss function becomes:

$$\tilde{J}(\theta; \mathbf{X}, \mathbf{y}) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + \frac{\alpha}{2} \sum_{j=1}^{n} \theta_j^2$$

Here, $h_\theta(x)$ is the hypothesis function, typically $h_\theta(x) = \theta^T x$ for linear regression.

3.4.7 Summary

L2 regularization helps improve the generalization of the model by penalizing large weights and thus preventing overfitting. It is widely used in various machine learning algorithms, including linear regression, logistic regression, neural networks, and support vector machines.

3.5 L1 Regularization

L1 regularization, also known as Lasso (Least Absolute Shrinkage and Selection Operator) when applied to linear models, is a technique used to prevent overfitting by adding a penalty term to the loss function. This penalty is the sum of the absolute values of the model parameters. The main objective of L1 regularization is to encourage sparsity in the model parameters, potentially setting some parameters exactly to zero.

3.5.1 Loss Function with L1 Regularization

In a typical machine learning model, we aim to minimize a loss function $J(\theta; \mathbf{X}, \mathbf{y})$ that measures the error of the model's predictions $\hat{y}$ compared to the true values $\mathbf{y}$. The model parameters are represented by θ.

With L1 regularization, the loss function is modified to include an additional term that penalizes large weights. The regularized loss function $\tilde{J}$ is given by:

$$\tilde{J}(\theta; \mathbf{X}, \mathbf{y}) = J(\theta; \mathbf{X}, \mathbf{y}) + \alpha \Omega(\theta)$$

where:

- $J(\theta; \mathbf{X}, \mathbf{y})$ is the original loss function.
- α is the regularization parameter that controls the strength of the regularization (a non-negative hyperparameter).
- $\Omega(\theta) = \sum_{j=1}^{n} |\theta_j|$ is the L1 penalty term (the sum of the absolute values of the model parameters).

3.5.2 Interpretation of the Penalty Term

The penalty term $\Omega(\theta)$ in L1 regularization is:

$$\Omega(\theta) = \sum_{j=1}^{n} |\theta_j|$$

This term adds the absolute values of all the model parameters θ_j. Unlike L2 regularization, which penalizes the square of the parameters, L1 regularization penalizes the absolute values, which can lead to some parameters being exactly zero, promoting sparsity.

3.5.3 Effect on Model Training

- **Preventing Overfitting:** By penalizing large values of θ, L1 regularization encourages the model to keep the parameters small, reducing the complexity of the model and helping to prevent overfitting.
- **Sparsity:** The L1 penalty can shrink some parameter estimates to exactly zero, effectively performing feature selection and creating a sparse model.

3.5.4 Gradient Descent with L1 Regularization

During gradient descent, the update rule for each parameter θ_j is modified to include the regularization term. The gradient of the regularized loss function with respect to θ_j involves the sign function due to the absolute value:

$$\frac{\partial \tilde{J}}{\partial \theta_j} = \frac{\partial J}{\partial \theta_j} + \alpha \operatorname{sign}(\theta_j)$$

where $\operatorname{sign}(\theta_j)$ is the sign function, defined as:

$$\operatorname{sign}(\theta_j) = \begin{cases} 1 & \text{if } \theta_j > 0 \\ -1 & \text{if } \theta_j < 0 \\ 0 & \text{if } \theta_j = 0 \end{cases}$$

This modifies the parameter update rule to:

$$\theta_j := \theta_j - \eta \left(\frac{\partial J}{\partial \theta_j} + \alpha \operatorname{sign}(\theta_j) \right)$$

where η is the learning rate. This update rule shows that each parameter θ_j is adjusted based not only on the gradient of the loss function but also shrunk towards zero by the term $\alpha \operatorname{sign}(\theta_j)$.

3.5.5 Choosing the Regularization Parameter α

- **Cross-Validation:** The value of α is typically chosen using cross-validation. It controls the trade-off between fitting the training data well and keeping the model parameters small.
- **Impact:** A larger α results in stronger regularization, leading to more parameters being shrunk to zero, potentially higher bias but lower variance. A smaller α results in weaker regularization, allowing the model to fit the training data more closely but with a higher risk of overfitting.

3.5.6 Example

Consider a linear regression model without regularization:

$$J(\theta; \mathbf{X}, \mathbf{y}) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2$$

With L1 regularization, the loss function becomes:

$$\tilde{J}(\theta; \mathbf{X}, \mathbf{y}) = \frac{1}{m} \sum_{i=1}^{m} (h_\theta(x^{(i)}) - y^{(i)})^2 + \alpha \sum_{j=1}^{n} |\theta_j|$$

Here, $h_\theta(x)$ is the hypothesis function, typically $h_\theta(x) = \theta^T x$ for linear regression.

3.5.7 Summary

L1 regularization helps improve the generalization of the model by penalizing large weights and promoting sparsity in the model parameters. It is widely used in various machine learning algorithms, including linear regression, logistic regression, neural networks, and support vector machines.

3.6 Norm Penalties as Constrained Optimization

Norm penalties in regularization can be interpreted as constrained optimization problems. This perspective helps in understanding how regularization techniques, such as L1 and L2 regularization, impose constraints on the model parameters to improve generalization and prevent overfitting.

3.6.1 Constrained Optimization Framework

Consider a typical unconstrained optimization problem where we aim to minimize a loss function $J(\theta; \mathbf{X}, \mathbf{y})$:

$$\min_\theta J(\theta; \mathbf{X}, \mathbf{y})$$

In regularized optimization, we add a penalty term to the loss function. For L2 regularization, the problem becomes:

$$\min_\theta \left(J(\theta; \mathbf{X}, \mathbf{y}) + \alpha \sum_{j=1}^{n} \theta_j^2 \right)$$

This can be interpreted as a constrained optimization problem where we minimize the loss function subject to a constraint on the L2 norm of the parameter vector θ:

$$\min_{\theta} J(\theta; \mathbf{X}, \mathbf{y}) \quad \text{subject to} \quad \|\theta\|_2^2 \leq c$$

Similarly, for L1 regularization, the problem is:

$$\min_{\theta} \left(J(\theta; \mathbf{X}, \mathbf{y}) + \alpha \sum_{j=1}^{n} |\theta_j| \right)$$

This can be interpreted as a constrained optimization problem with a constraint on the L1 norm of the parameter vector θ:

$$\min_{\theta} J(\theta; \mathbf{X}, \mathbf{y}) \quad \text{subject to} \quad \|\theta\|_1 \leq c$$

3.6.2 Equivalence of Penalized and Constrained Forms

The penalized form of the optimization problem with norm penalties can be converted to its equivalent constrained form by adjusting the constraint bounds. Here, we illustrate the equivalence for both L1 and L2 regularization.

L2 Regularization as Constrained Optimization

For L2 regularization, the penalized form is:

$$\min_{\theta} \left(J(\theta; \mathbf{X}, \mathbf{y}) + \alpha \|\theta\|_2^2 \right)$$

This is equivalent to the constrained form:

$$\min_{\theta} J(\theta; \mathbf{X}, \mathbf{y}) \quad \text{subject to} \quad \|\theta\|_2^2 \leq c$$

where c is a positive constant. The value of α determines the tightness of the constraint c. A larger α corresponds to a smaller c, enforcing a stricter constraint on the parameter norm.

L1 Regularization as Constrained Optimization

For L1 regularization, the penalized form is:

$$\min_{\theta} \left(J(\theta; \mathbf{X}, \mathbf{y}) + \alpha \|\theta\|_1 \right)$$

This is equivalent to the constrained form:

$$\min_{\theta} J(\theta; \mathbf{X}, \mathbf{y}) \quad \text{subject to} \quad \|\theta\|_1 \leq c$$

where c is a positive constant. The value of α determines the tightness of the constraint c. A larger α corresponds to a smaller c, enforcing a stricter constraint on the parameter norm.

3.6.3 Interpretation and Benefits

- **Improved Generalization:** By constraining the norm of the parameters, we prevent the model from fitting the noise in the training data, leading to better generalization on unseen data.
- **Feature Selection:** L1 regularization (Lasso) can set some parameter values to exactly zero, effectively performing feature selection and simplifying the model.
- **Stability:** Constraining the parameter norms can make the optimization problem more stable and less sensitive to small changes in the data.

3.6.4 Example: Linear Regression with L2 Regularization

Consider a linear regression model without regularization:

$$J(\theta;\mathbf{X},\mathbf{y}) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

With L2 regularization, the penalized form is:

$$\tilde{J}(\theta;\mathbf{X},\mathbf{y}) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \alpha\|\theta\|_2^2$$

This is equivalent to the constrained optimization problem:

$$\min_\theta J(\theta;\mathbf{X},\mathbf{y}) \quad \text{subject to} \quad \|\theta\|_2^2 \leq c$$

where c is determined by the regularization parameter α.

3.6.5 Example: Linear Regression with L1 Regularization

Consider a linear regression model without regularization:

$$J(\theta;\mathbf{X},\mathbf{y}) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2$$

With L1 regularization, the penalized form is:

$$\tilde{J}(\theta;\mathbf{X},\mathbf{y}) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \alpha\|\theta\|_1$$

This is equivalent to the constrained optimization problem:

$$\min_\theta J(\theta;\mathbf{X},\mathbf{y}) \quad \text{subject to} \quad \|\theta\|_1 \leq c$$

where c is determined by the regularization parameter α.

In summary, norm penalties can be viewed as constrained optimization problems where the model parameters are restricted to lie within a feasible set defined by their norms. This interpretation provides insight into how regularization techniques improve model generalization and robustness.

3.7 Regularized & Under-constrained Problems

Underdetermined problems have infinitely many solutions. Regularization is essential in these cases to ensure stability and generalization. An example is using the Moore-Penrose pseudoinverse with L2-regularization:

$$(X'X + \alpha I)^{-1}X'y$$

Dataset Augmentation: Data segmentation stands as a crucial process in this landscape, facilitating the organization and analysis of datasets to derive meaningful insights. From enhancing model accuracy to optimizing decision-making processes, data segmentation plays a pivotal role.

Data segmentation is the process of breaking down a dataset into discrete groups according to specific standards or attributes. These subsets can be identified by several criteria, including behavior, demographics, or certain dataset features. The main goal of data segmentation is to enable more focused analysis and modeling to produce better results.

3.7.1 Importance of Data Segmentation

Segmentation plays a critical role in machine learning by enhancing the quality of data analysis and model performance. Here's why segmentation is important in the context of machine learning:

- **Improved Model Accuracy:** Segmentation allows machine learning models to focus on specific subsets of data, often leading to more accurate predictions or classifications. By training models on segmented data, they can capture nuances and patterns specific to each segment, resulting in better overall performance.
- **Enhanced Understanding:** Segmentation helps comprehend the data's underlying structure on a deeper level. By grouping data into meaningful categories, analysts can uncover hidden patterns, correlations, and trends that may not be visible when examining the data as a whole. This deeper understanding can aid in strategy formulation and decision-making.
- **Customized Solutions:** Segmentation enables the creation of strategies and solutions tailored to specific dataset segments. Personalized techniques significantly improve outcomes in various industries, including marketing, healthcare, and finance. For example, in healthcare, segmented patient data allows for customized treatment plans and illness management strategies.
- **Optimized Resource Allocation:** By segmenting data, organizations can allocate resources more efficiently. In marketing campaigns, for instance, targeting specific customer segments with tailored messages or offers can maximize the return on investment by focusing resources where they are most likely to yield results.
- **Effective Risk Management:** Segmentation aids in identifying high-risk segments within a dataset, enabling proactive risk assessment and mitigation strategies. This is particularly crucial in fields like finance and insurance, where accurately assessing risk can prevent financial losses.

In summary, data segmentation is a powerful tool in deep learning and machine learning, enhancing model accuracy, understanding, customized solutions, resource allocation, and risk management.

3.8 Noise Robustness

Noise robustness refers to the ability of a model to maintain its performance despite the presence of noise during training and/or inference. Noise-robust models generalize better and are less prone to overfitting. Adding noise to inputs or weights during training acts as a regularizer, often more effective than merely shrinking parameters, and can be seen as Bayesian inference over the weights.

Noise can come from intrinsic variability in real-world data or errors in data transmission and processing. Models lacking noise robustness tend to fail when encountering previously unknown noisy patterns.

Robust models maintain high performance levels even in noisy conditions because they rely more on relevant patterns and less on coincidental correlations. Enhancing noise robustness helps prevent overfitting and improves a model's generalization ability on diverse real-world data.

3.8.1 Benefits of Noise Robustness

- **Improved Generalization:** Noise-robust models can generalize better to unseen data, as they are less likely to overfit to the noise in the training data.
- **Enhanced Stability:** Models with noise robustness are more stable and perform consistently under varying conditions, including those with noise.
- **Increased Reliability:** These models are more reliable in real-world applications, where noise is often unavoidable.

3.9 Semi-Supervised Learning

In deep learning, regularization techniques are employed to prevent overfitting and improve the generalization of the model. Traditional regularization methods like L1 and L2 regularization penalize the magnitude of weights in the network. Semi-supervised learning can serve as an alternative regularization technique by leveraging both labeled and unlabeled data during training.

When training a deep learning model with semi-supervised learning, the model is exposed to both labeled and unlabeled data. The labeled data helps in learning the task-specific features, while the unlabeled data provides additional information about the underlying data distribution.

By incorporating unlabeled data, semi-supervised learning encourages the model to learn more robust and generalizable representations of the input data. This additional regularization helps prevent overfitting, especially in scenarios where labeled data is scarce or expensive to acquire.

Semi-supervised learning offers several benefits:

- **Enhanced Generalization:** Using unlabeled data helps the model capture the underlying data distribution, leading to better generalization.
- **Reduced Overfitting:** The incorporation of unlabeled data acts as a regularizer, preventing the model from overfitting to the labeled data.
- **Efficient Use of Data:** Semi-supervised learning maximizes the utility of available data, making it particularly valuable when labeled data is limited or costly.

3.10 Multi-Task Learning

Multi-task learning involves training a single model to perform multiple tasks simultaneously. Instead of training separate models for each task, multi-task learning shares information across tasks, leading to improved generalization and regularization.

In deep learning, multi-task learning can be implemented by modifying the architecture of the neural network to include multiple output layers, each corresponding to a different task. By jointly optimizing these output layers along with the shared layers of the network, the model learns to extract features that are useful for all tasks.

By learning multiple tasks simultaneously, multi-task learning encourages the model to discover common underlying patterns in the data, which can help in regularizing the model and improving its performance on each individual task.

- **Shared Representation:** Common layers in the network learn representations that are useful for all tasks, enhancing the model's ability to generalize.
- **Task-Specific Outputs:** Separate output layers allow the model to fine-tune its predictions for each specific task.
- **Improved Regularization:** By learning from multiple tasks, the model can avoid overfitting to any single task, leading to better overall performance.

3.11 Early Stopping of Training

Early stopping is a regularization technique used to prevent overfitting in machine learning and deep learning models. It works by halting the training process when the model's performance on a validation dataset begins to degrade. This method effectively determines the optimal amount of training and regularization automatically.

When a model is trained for too many epochs, it may start to memorize the training data instead of learning generalizable patterns. This leads to overfitting, where the model performs well on the training data but poorly on unseen data. Early stopping mitigates overfitting by stopping the training process before the model starts to overfit.

The key idea behind early stopping is to monitor the model's performance on a validation dataset at regular intervals during training. If the validation loss stops decreasing or starts to increase consistently for a specified number of epochs (known as the patience parameter), early stopping is triggered, and training is halted. The model parameters at the point of early stopping are then used as the final model.

Early stopping is a simple yet effective technique that helps prevent overfitting without introducing additional hyperparameters or significant computational overhead.

3.11.1 How Early Stopping Works

- **Monitor Performance:** During training, the model's performance on a separate validation dataset is monitored after each epoch.
- **Patience Parameter:** Define a patience parameter that specifies the number of epochs to wait for an improvement in validation loss.
- **Triggering Early Stopping:** If the validation loss does not decrease for the specified number of epochs (patience), stop the training process.
- **Final Model:** Use the model parameters at the point of early stopping as the final model.

3.11.2 Example

Consider training a neural network for image classification. The dataset is split into training and validation sets. During training, we monitor the validation loss after each epoch. If the validation loss does not decrease for a certain number of epochs (patience), we stop training and use the model parameters at the point of early stopping.

3.12 Bagging

Bagging (Bootstrap Aggregating) is a popular ensemble method used to improve the stability and generalization of machine learning models, thereby serving as a regularization technique. Bagging works by training multiple base models on different subsets of the training data and then combining their predictions to make the final prediction.

Each base model is trained on a randomly sampled subset of the training data with replacement (bootstrap sampling). By training multiple models on different subsets of data, bagging reduces the variance of the predictions and helps prevent overfitting.

Solution for Overfitting Using Bagging

To address overfitting using bagging, follow these steps:

1. **Split the Dataset**: Split the original dataset into a training set and a validation set.

2. **Apply Bagging**: Apply bagging by training multiple base models on different bootstrap samples of the training set. Each base model should use a different subset of the training data.

3. **Combine Predictions**: Make predictions using each base model on the validation set. Then, combine the predictions of all base models, typically by averaging for regression tasks or by voting for classification tasks.

4. **Evaluate Performance**: Evaluate the performance of the bagging ensemble on the validation set. This step helps in assessing whether bagging effectively reduces overfitting compared to training a single model.

5. **Adjust Hyperparameters**: Fine-tune the hyperparameters of the base models and the bagging ensemble, if necessary, to further improve performance.

3.13 Dropout

Dropout is a regularization technique used to prevent overfitting in neural networks by randomly dropping out units (hidden and visible) during training. This method enhances the robustness of the network by preventing it from relying too much on any individual neuron.

During each training iteration, a fraction of neurons is randomly selected and temporarily removed from the network, along with all of their incoming and outgoing connections. This forces the network to learn more diverse and robust features, as different subsets of neurons are active in each iteration.

By randomly dropping out neurons, dropout effectively trains an ensemble of multiple neural networks with shared weights. This ensemble approach helps prevent overfitting by promoting generalization.

At test time, all neurons are used, but their weights are scaled down by the dropout rate to account for the fact that more units are active during training than during testing.

Implementation of Dropout

To apply dropout as a solution to overfitting in a neural network, dropout layers can be added after each hidden layer during training. The dropout rate specifies the fraction of neurons to drop out during each training iteration.

3.14 Adversarial Training

Adversarial Training as a Regularization Technique

Adversarial training involves augmenting the training data with adversarial examples generated by adding small perturbations to input samples. These perturbations are crafted to intentionally mislead the model without being perceivable to humans. By exposing the model to these adversarial examples during training, it learns to become more robust and resistant to such attacks, thus improving its generalization performance.

During adversarial training, the model is trained to not only minimize the loss on the original training examples but also to maximize the loss on the adversarial examples. This encourages the model to learn features that are robust and invariant to small perturbations in the input space.

Solution for Overfitting with Adversarial Training

Adversarial training can also act as a solution for overfitting by encouraging the model to learn more generalizable features. Since adversarial examples are generated to be close to the decision boundary of the model, training with these examples forces the model to learn a smoother decision boundary that generalizes better to unseen data.

Additionally, adversarial training can help in regularizing the model by implicitly applying a form of data augmentation. By generating adversarial examples from the original training data, the model is exposed to a more diverse set of examples, which helps prevent overfitting by introducing more variability into the training process.

3.15 Parameter Tying and Sharing

Overfitting occurs when a model memorizes the training data instead of generalizing to unseen data. Regularization techniques like Parameter Tying and Parameter Sharing combat overfitting by introducing constraints or sharing parameters, preventing the model from becoming overly complex and memorizing noise in the training data.

By constraining or sharing parameters across different parts of the model, these regularization techniques encourage the model to learn simpler and more interpretable representations of the input

data. This reduces the risk of overfitting and improves the model's ability to generalize to unseen data.

Parameter sharing, where models share parameters, not only reduces memory usage but also enhances generalization. This is commonly seen in convolutional neural networks, where the same parameters detect features across different regions of an image.

3.16 Sparse Representations

Sparse representation encourages the model to use only a small subset of features during training, leading to simpler and more interpretable models. This regularization technique penalizes the magnitudes of the model parameters, promoting sparsity in the learned representations.

In machine learning, sparse representation can be achieved through techniques such as L1 regularization (Lasso regression) or by directly optimizing for sparsity using techniques like LARS (Least Angle Regression) or coordinate descent.

By encouraging sparsity in the learned representations, sparse representation helps prevent overfitting by reducing the complexity of the model and focusing on the most informative features.

3.17 Tangent Distance, Tangent Prop, and Manifold Tangent Classifier

In the realm of machine learning, techniques like Tangent Distance, Tangent Propagation (Tangent Prop), and the Manifold Tangent Classifier are employed to harness the notion of data residing on low-dimensional manifolds. These methods aim to ensure that minor perturbations along the manifold do not significantly alter the predictions of the model.

3.17.1 Understanding Manifold Space

In deep learning, the concept of manifold space refers to the underlying low-dimensional structure where high-dimensional data points lie. Despite residing in high-dimensional spaces, many real-world datasets exhibit intrinsic low-dimensional characteristics. By grasping and leveraging this manifold structure, more efficient and effective learning algorithms can be devised.

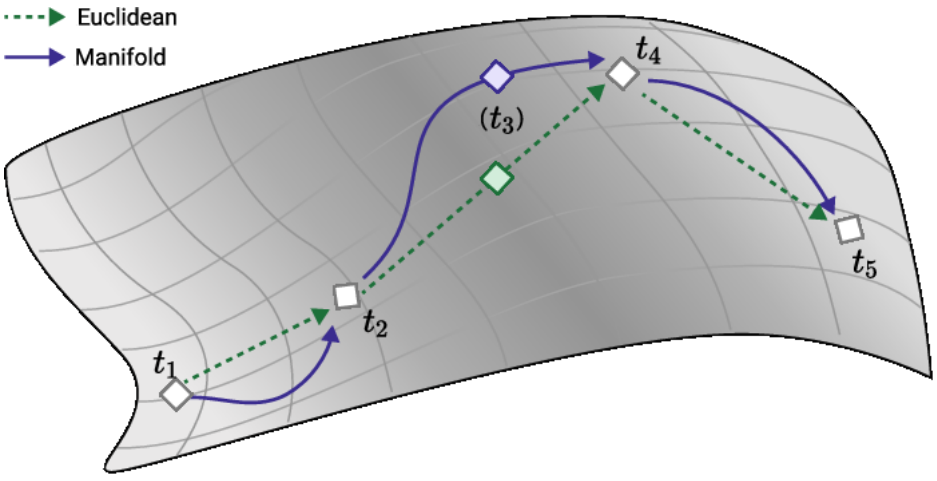


Figure 3.1: Visualization depicting data points on a low-dimensional manifold

3.18 Tangent Distance for Regularization

3.18.1 Tangent Distance

Tangent Distance is a metric used to gauge the similarity between two points on a manifold by considering their respective tangent vectors. Particularly useful in deep learning, this approach is instrumental in tasks where data resides on a low-dimensional manifold within a higher-dimensional space.

Key Concepts

- **Manifold Assumption**: Data points adhere to a smooth manifold $\mathscr{M}$ embedded within a higher-dimensional space.
- **Tangent Vectors**: At each data point, the tangent space $T_x\mathscr{M}$ serves as a linear approximation of the local manifold structure.
- **Tangent Distance**: The distance between two points on the manifold is estimated by measuring the distance between their projections onto the tangent spaces.

Utilization in Regularization

Tangent Distance can be utilized for regularization purposes by ensuring that the learned representation aligns with the manifold structure of the data. This is achieved by integrating the tangent distance into the loss function, compelling the model to learn features that remain invariant to minor perturbations along the manifold.

Example: Tangent Distance in Deep Learning

Consider an image classification scenario where images of digits are situated on a manifold within a high-dimensional pixel space. To compute the Tangent Distance between two images:

1. Compute the tangent vectors at each image.
2. Project the images onto their respective tangent spaces.
3. Measure the distance between these projections.

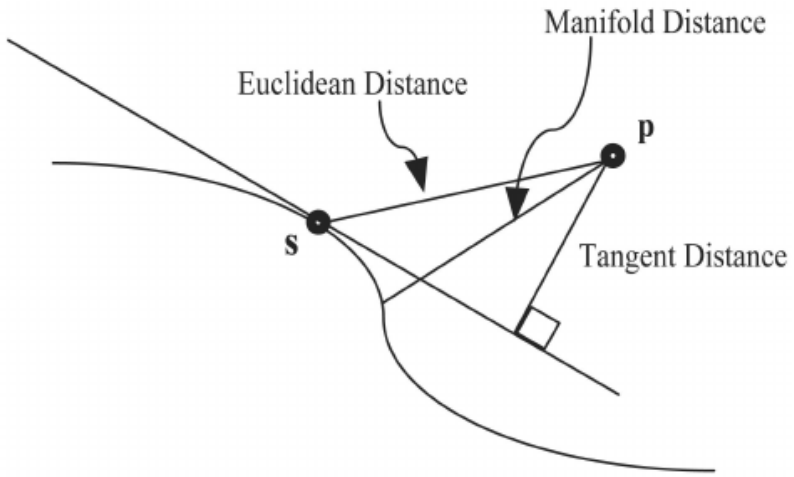


Figure 3.2: Tangent Distance: Projection of points onto their respective tangent spaces.

3.19 Tangent Propagation for Robustness

Tangent Propagation (Tangent Prop) is a technique employed to enforce local invariance to minor transformations by penalizing the directional derivatives of the network output concerning input perturbations. It encourages the model to exhibit invariance to specific transformations characterized by tangent vectors at each data point on the manifold.

3.19.1 Conceptual Framework

- **Manifold Assumption**: Data points either reside on or close to a low-dimensional manifold embedded in a higher-dimensional space.
- **Tangent Vectors**: Minor transformations can be approximated by tangent vectors at each data point.
- **Regularization Term**: A penalty term is incorporated into the loss function to minimize the sensitivity of the model output concerning these tangent vectors.

3.20 Manifold Tangent Classifier for Improved Classification

The Manifold Tangent Classifier is a method that leverages tangent vectors of the data manifold to enhance classification performance. It aims to make the classifier invariant to minor transformations of the input data by utilizing tangent vectors to represent such alterations.

3.20.1 Key Components

- **Manifold Assumption**: Data points either reside on or close to a smooth manifold $\mathscr{M}$ embedded in a higher-dimensional space.
- **Tangent Vectors**: At each data point, the tangent space $T_x\mathscr{M}$ encompasses vectors representing minor, local transformations.
- **Classifier Invariance**: The classifier is trained to exhibit invariance to these minor transformations by integrating tangent vectors into the classification process.

3.20.2 Implementation Steps

1. **Compute Tangent Vectors**: Calculate tangent vectors for each data point to approximate local transformations.
2. **Augment Data**: Utilize tangent vectors to generate new data points representing the original points perturbed along the tangent directions.
3. **Train Classifier**: Train a classifier using both original and augmented data to ensure invariance to minor transformations.

IV

Unit IV

4 Optimization for Training Deep Models 71

4.1 Understanding the Difference: Learning vs. Pure Optimization
4.2 Visualizing the Differences
4.3 Challenges in Neural Network Optimization
4.4 Basic Optimization Algorithms
4.5 Parameter Initialization Strategies
4.6 Algorithms with Adaptive Learning Rates
4.7 Approximate Second-Order Methods in Deep Learning
4.8 Optimization Strategies and Meta-Algorithms

4. Optimization for Training Deep Models

4.1 Understanding the Difference: Learning vs. Pure Optimization

In the realm of optimizing deep models, it's crucial to grasp the distinction between learning in machine learning (ML) and pure optimization.

4.1.1 Machine Learning Perspective

In machine learning, our primary objective is to enhance a performance measure P, such as accuracy, on a test set. To achieve this, we delve into optimizing an objective function $J(\theta)$, typically a measure like cross-entropy loss, with the hope that this optimization also boosts P.

4.1.2 Pure Optimization Angle

On the flip side, pure optimization focuses solely on optimizing $J(\theta)$. There's no separate performance measure involved; the optimization of $J(\theta)$ is the ultimate goal.

Understanding Expected Generalization Error

The concept of expected generalization error lies at the heart of this comparison.

- In machine learning, our aim is to minimize the expected generalization error (also known as risk) across the true data-generating distribution p_{data}. If we had direct access to p_{data}, it would boil down to a pure optimization problem.
- However, in practical scenarios, we often lack access to p_{data} and operate with a finite training set instead. This leads us to approximate p_{data} with the empirical distribution derived from the training set, thus giving rise to the concept of empirical risk minimization (ERM).

4.1.3 Understanding Empirical Risk Minimization (ERM)

Empirical Risk Minimization (ERM) serves as a guiding principle in machine learning to approximate the expected risk when we're limited to a finite set of training data.

4.1.4 Expected Risk

The expected risk, or generalization error, quantifies the average loss over the entire data-generating distribution p_{data}. It's mathematically represented as:

$$R(\theta) = \mathbb{E}_{(x,y)\sim p_{\text{data}}}[L(f_\theta(x), y)],$$

where L is the loss function, f_θ represents the model parameterized by θ, and (x, y) denote input-output pairs.

Empirical Risk

Given that p_{data} is typically unknown, we resort to approximating the expected risk using empirical risk, which computes the average loss over a finite training set $\{(x_i, y_i)\}_{i=1}^n$:

$$R_{\text{emp}}(\theta) = \frac{1}{n}\sum_{i=1}^{n} L(f_\theta(x_i), y_i).$$

The Essence of ERM

Empirical Risk Minimization (ERM) entails finding the model parameters θ that minimize the empirical risk:

$$\hat{\theta} = \arg\min_\theta R_{\text{emp}}(\theta).$$

This approach operates under the assumption that minimizing the empirical risk leads to favorable generalization on unseen data.

Key Components and Challenges of ERM

To fully grasp ERM, it's essential to delve into its key components and associated challenges.

- **Training Set**: Comprises a finite set of examples drawn from the true distribution.
- **Loss Function**: Measures the deviation between the model's predictions and the actual labels.
- **Model Parameters**: Represent the parameters θ defining the model f_θ.
- **Challenges**: ERM can face hurdles such as overfitting, where aggressive minimization of empirical risk results in poor generalization, and the ultimate goal remains achieving robust generalization performance.

4.2 Visualizing the Differences

A diagrammatic representation helps solidify the disparities between learning and pure optimization.

This visualization encapsulates the contrasting methodologies employed in machine learning versus pure optimization

4.3 Challenges in Neural Network Optimization

The optimization of neural networks involves several challenges, particularly due to the complex, high-dimensional loss landscapes. Below, we explore some of these challenges in detail.

4.3.1 Ill-conditioning

Ill-conditioning in the context of optimization and numerical methods refers to problems where small changes in input can cause large changes in the output. This makes the problem difficult to solve accurately.

Basic Concept Imagine you're trying to solve a problem where you want to find the best-fit line for a set of data points. If your data points are almost perfectly aligned in a straight line, but not

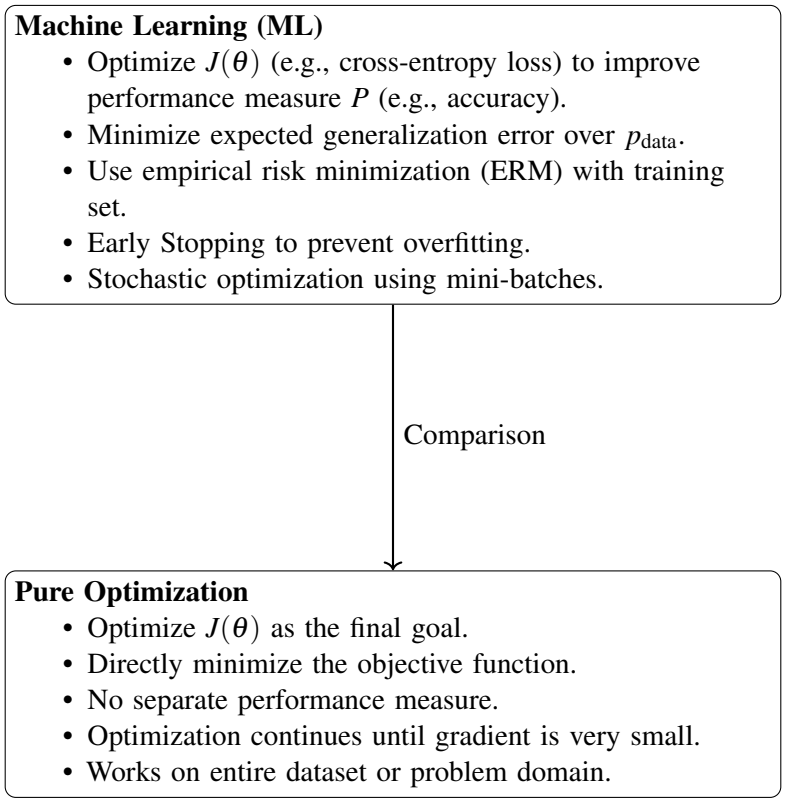


Figure 4.1: Differences between Learning and Pure Optimization

exactly, finding the precise best-fit line can be very sensitive to small changes in the data points. This is an example of ill-conditioning.

Example for Beginners Let's say you have two nearly parallel lines and you want to find their intersection. If the lines are exactly parallel, they never intersect. But if they are very close to being parallel, their intersection point will be very far away and sensitive to small changes in the slope of either line. Even a tiny adjustment can make a big difference in where they intersect. This makes the problem ill-conditioned.

Key Points Sensitivity to Input Changes:

In ill-conditioned problems, small changes or errors in input data can cause large changes in the output. This makes it hard to get accurate results. Numerical Instability:

Because of this sensitivity, numerical algorithms may produce inaccurate results due to rounding errors or approximations. Condition Number:

One way to measure ill-conditioning is by using the condition number. A high condition number indicates a more ill-conditioned problem.

The Hessian matrix is a square matrix of second-order partial derivatives of a scalar-valued function. It is used extensively in optimization and numerical analysis to understand the curvature of the function being optimized.

The **Hessian matrix** H is defined as:

$$H = \nabla^2 J(\theta),$$

where $J(\theta)$ is the cost function and θ represents the model parameters. The condition number of H is defined as:

$$\kappa(H) = \frac{\lambda_{\max}(H)}{\lambda_{\min}(H)},$$

where λ_{max} and λ_{min} are the largest and smallest eigenvalues of H. A high condition number indicates that the Hessian is ill-conditioned, meaning that the optimization process is highly sensitive to changes in the input. This can lead to scenarios where even very small steps may increase the cost function, causing the gradient descent to get stuck.

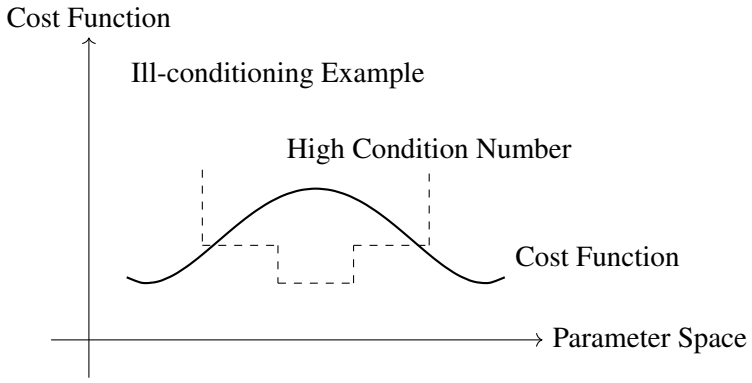


Figure 4.2: Ill-conditioned Hessian causing optimization steps to increase cost function

4.3.2 Local Minima

Nearly any deep learning model is guaranteed to have an extremely large number of local minima due to the **model identifiability problem**. A model is said to be identifiable if a sufficiently large training set can rule out all but one setting of the model parameters. In the case of neural networks, we can obtain equivalent models by swapping the positions of the neurons, making them non-identifiable.

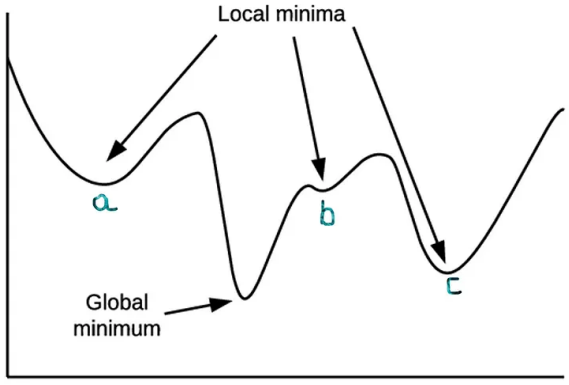


Figure 4.3: Local Minima

4.3.3 Plateaus, Saddle Points, and Other Flat Regions

A **saddle point** is a point where the gradient is zero, but it is not a local minimum or maximum. At a saddle point, some directions in the parameter space lead to increasing values of the cost function, while others lead to decreasing values. Plateaus are regions where the gradient is very small, causing slow progress in optimization.

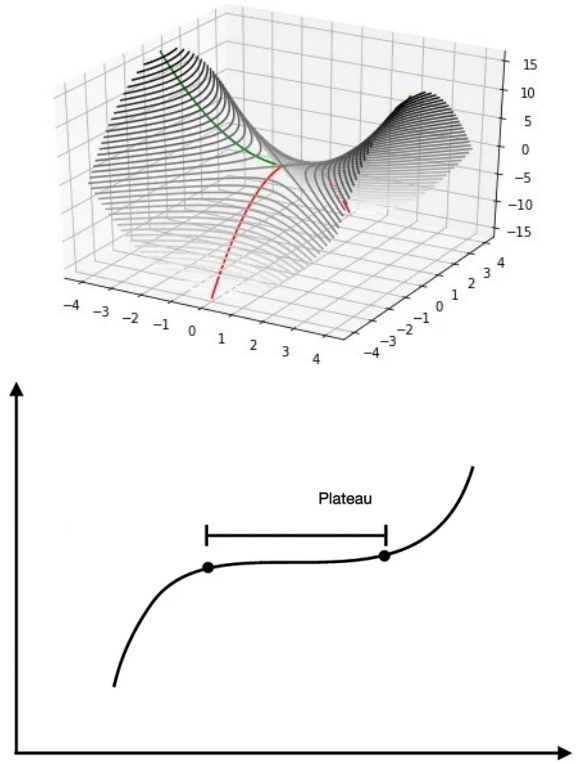


Figure 4.4: Saddle Point

4.3.4 Cliffs and Exploding Gradients

Neural networks might sometimes have extremely steep regions, resembling cliffs. For example, consider a 3-layer neural network with linear activation functions and the same weight matrix W used in each layer. The output layer y is given by:

$$y = W \cdot (W \cdot (W \cdot x)) = W^3 x.$$

This repeated multiplication can lead to sharp non-linearities and steep gradients, causing the gradient descent to take very large steps, known as exploding gradients.

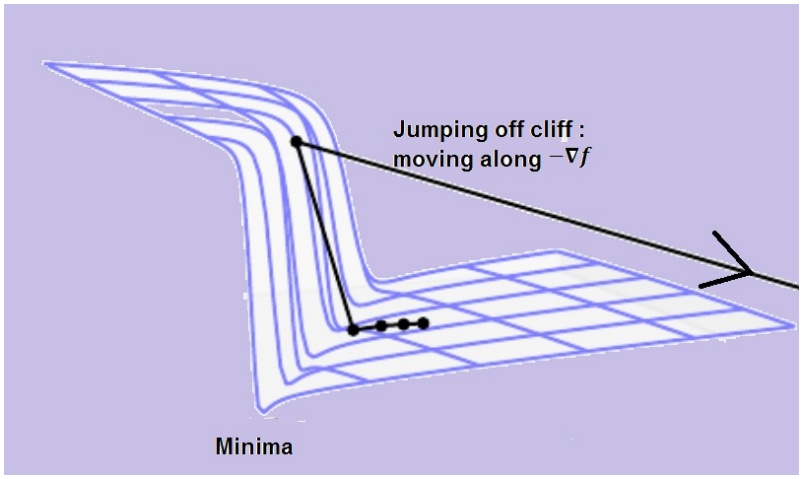


Figure 4.5: Cliffs and Exploding Gradients

4.3.5 Long-Term Dependencies

When a neural network becomes sufficiently deep, such as in recurrent neural networks (RNNs), long-term dependencies can cause problems. If the same weight matrix W is used repeatedly, after t steps, we get:

$$h_t = W^t h_0.$$

This can lead to either vanishing gradients (when $\|W\| < 1$) or exploding gradients (when $\|W\| > 1$).

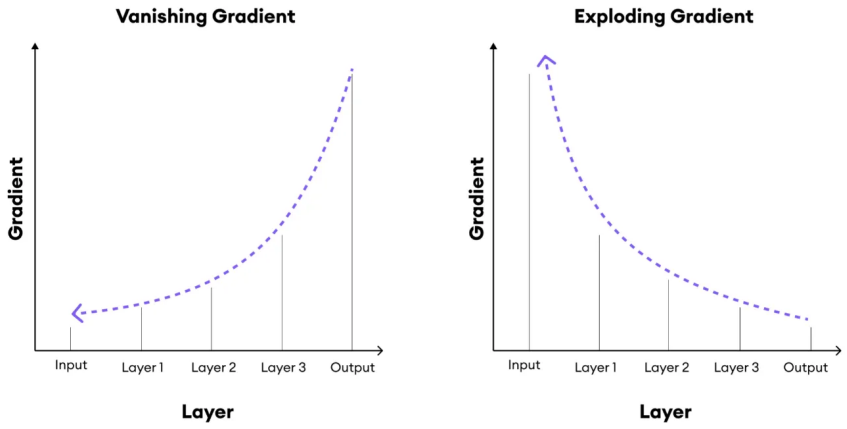


Figure 4.6: Vanishing and Exploding Gradients in Deep Networks

4.3.6 Weight Space Symmetry

Weight space symmetry refers to the fact that the same result can be obtained by swapping the weights between units. This is known as weight space symmetry and can be described by the number of permutations of the hidden units. For n hidden units, there are $n!$ permutations.

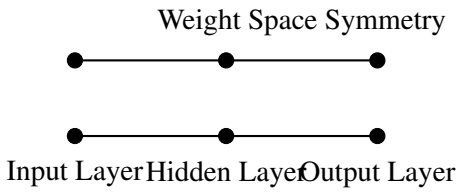


Figure 4.7: Weight Space Symmetry in Neural Networks

4.4 Basic Optimization Algorithms

In deep learning, several optimization algorithms are employed to efficiently train neural networks. Here, we'll discuss some fundamental algorithms: Gradient Descent, Stochastic Gradient Descent (SGD), Adam, and Momentum.

4.4.1 Gradient Descent Algorithm

Gradient Descent (GD) is a widely used optimization algorithm that minimizes the cost function of a neural network model during training. It works by iteratively adjusting the model parameters in the direction of the negative gradient of the cost function until convergence.

Algorithm Steps:

1. **Initialization:** Initialize the model parameters with random values.
2. **Compute Gradient:** Calculate the gradient of the loss function with respect to the parameters.
3. **Update Parameters:** Adjust the parameters using the gradient and a predefined learning rate.
4. **Repeat:** Iterate until convergence or a maximum number of iterations is reached.

Weight Update Equation:

$$\theta = \theta - \alpha \nabla J(\theta)$$

4.4.2 Stochastic Gradient Descent (SGD)

Stochastic Gradient Descent (SGD) is a variant of Gradient Descent used for optimizing machine learning models, particularly in scenarios with large datasets.

Algorithm 1 Stochastic Gradient Descent

Input: Training data D, learning rate α, maximum iterations T
Initialize: Randomly initialize model parameters θ
for $t = 1$ to T **do**
    Randomly shuffle training data D
    for each training example $(x^{(i)}, y^{(i)})$ in D **do**
        Compute gradient: $\nabla J(\theta; x^{(i)}, y^{(i)})$
        Update parameters: $\theta \leftarrow \theta - \alpha \nabla J(\theta; x^{(i)}, y^{(i)})$
    end for
end for
Output: Optimized parameters θ

4.4.3 Adam

Adam (Adaptive Moment Estimation) is an adaptive optimization algorithm that computes adaptive learning rates for different parameters based on estimates of first and second moments of the gradients.

Parameter Update Equations:

$$m = \beta_1 m + (1 - \beta_1) \nabla J(\theta)$$
$$v = \beta_2 v + (1 - \beta_2) (\nabla J(\theta))^2$$
$$\theta = \theta - \frac{\alpha}{\sqrt{v} + \varepsilon} \cdot m$$

4.4.4 Momentum

Momentum is an optimization algorithm that accumulates the exponentially decaying moving average of past gradients (velocity) and uses it to determine the direction of the next step.

Update Equations:

$$v_t = \alpha v_{t-1} + \eta \nabla J(\theta)$$
$$\theta = \theta - v_t$$

where α is the momentum parameter, η is the learning rate, and $\nabla J(\theta)$ is the gradient of the cost function with respect to the parameters θ.

4.5 Parameter Initialization Strategies

When embarking on the training journey of a deep learning model, one crucial aspect to consider is the initialization of its parameters. This initialization step sets the starting point for the iterative optimization process, influencing whether the algorithm converges effectively and to what quality of solution.

A fundamental principle guiding parameter initialization is the necessity to break symmetry within the network. Symmetry breaking ensures that each neuron learns distinct features from the input data. Without this distinction, multiple neurons connected to the same input would update identically, hindering the network's ability to extract meaningful information.

To achieve symmetry breaking, weights are typically randomly initialized, often drawn from Gaussian or uniform distributions. This randomness promotes diversity in neuron activations and is computationally efficient compared to deterministic initialization methods.

While weights are randomized, biases are often set heuristically, with zero being a common choice. This asymmetry complements the randomized weights, further aiding in breaking symmetry within the network.

However, the scale of these initializations is critical. Large weights may effectively break symmetry but can lead to instability during training, causing issues like exploding gradients. Conversely, excessively small weights may result in vanishing gradients, impeding the flow of information through the network.

To mitigate these challenges, careful consideration of the initialization scale is necessary. Techniques such as Xavier and He initialization offer principled approaches to determining suitable weight scales, ensuring stable and efficient training.

4.6 Algorithms with Adaptive Learning Rates

Adaptive learning rate algorithms play a pivotal role in optimizing the training process of deep neural networks by dynamically adjusting learning rates based on the gradient history. Three prominent algorithms in this category are AdaGrad, RMSProp, and Adam.

4.6.1 AdaGrad

AdaGrad adapts the learning rate for each parameter based on the historical gradients. It scales the learning rates inversely proportional to the square root of the sum of squared gradients, effectively decaying the learning rate for parameters experiencing frequent updates. The parameter update equation for AdaGrad is given by:

$$\theta = \theta - \frac{\alpha}{\sqrt{r+\varepsilon}} \cdot \nabla J(\theta)$$

where θ represents the parameter being updated, α denotes the learning rate, r accumulates the squared gradients over time, and ε prevents division by zero.

4.6.2 RMSProp

RMSProp addresses the issue of overly aggressive learning rate decay in AdaGrad by employing an exponentially weighted moving average for the gradient accumulation. This modification ensures more stable updates, particularly in non-convex optimization landscapes. The update equation for RMSProp is similar to AdaGrad, with the inclusion of a decay rate parameter β:

$$\theta = \theta - \frac{\alpha}{\sqrt{r+\varepsilon}} \cdot \nabla J(\theta)$$

where β controls the decay rate of the moving average.

4.6.3 Adam

Adam combines the benefits of momentum optimization and RMSProp by maintaining both a decaying average of past gradients and squared gradients. This combination allows Adam to adaptively adjust learning rates for each parameter while incorporating momentum to accelerate convergence. The update equations for Adam are:

$$m = \beta_1 m + (1 - \beta_1)\nabla J(\theta)$$

$$v = \beta_2 v + (1 - \beta_2)(\nabla J(\theta))^2$$

$$\theta = \theta - \frac{\alpha}{\sqrt{v} + \varepsilon} \cdot m$$

where β_1 and β_2 are decay rate parameters for the first and second moments, respectively.

These adaptive learning rate algorithms alleviate the need for manual tuning of learning rates, leading to faster convergence and improved performance in training deep neural networks.

4.7 Approximate Second-Order Methods in Deep Learning

Approximate second-order optimization methods aim to capture curvature information of the cost function to enhance convergence speed and stability compared to first-order methods like Gradient Descent. Three notable approaches in this category are the Gauss-Newton method, Limited-Memory BFGS (L-BFGS), and Natural Gradient Descent.

4.7.1 Gauss-Newton Method

The Gauss-Newton method approximates the Hessian matrix of the cost function using the outer product of gradients. By inversely scaling the Hessian approximation, it computes parameter updates that account for curvature information. However, exact computation of the Hessian can be computationally prohibitive, especially for large neural networks.

4.7.2 Limited-Memory BFGS (L-BFGS)

L-BFGS offers a memory-efficient approximation of the BFGS algorithm by storing a limited number of vectors to represent the curvature of the cost function. This approach balances computational complexity and convergence properties, making it well-suited for optimizing deep neural networks.

4.7.3 Natural Gradient Descent

Natural Gradient Descent incorporates the geometry of the parameter space by scaling gradients with the inverse Fisher Information Matrix. This adaptation allows for more informed updates, particularly in high-dimensional spaces, potentially accelerating convergence compared to standard Gradient Descent methods.

These approximate second-order methods provide alternatives to traditional first-order optimization techniques, offering improved convergence properties and efficiency in training deep neural networks.

4.8 Optimization Strategies and Meta-Algorithms

Optimization strategies and meta-algorithms are essential components of deep learning frameworks, enabling efficient training and improved generalization. These techniques address challenges such as slow convergence, overfitting, and oscillations in the optimization process.

4.8.1 Optimization Strategies

Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, and Adaptive Learning Rate Methods are foundational optimization strategies in deep learning:

1. SGD updates model parameters using gradients computed from random subsets of training examples, making it computationally efficient.
2. Mini-Batch Gradient Descent balances the efficiency of SGD with the stability of Batch Gradient Descent by updating parameters using mini-batches of examples.
3. Adaptive Learning Rate Methods dynamically adjust learning rates based on gradient history, facilitating faster convergence and improved performance.

4.8.2 Meta-Algorithms

Meta-algorithms complement optimization strategies and contribute to robust and effective training:

1. Ensemble Methods combine multiple models to enhance prediction accuracy and generalization, leveraging techniques like Bagging, Boosting, and Stacking.
2. Early Stopping prevents overfitting by halting training when performance on a validation set deteriorates, promoting better generalization.
3. Dropout is a regularization technique that mitigates overfitting by randomly deactivating neurons during training, encouraging robustness and generalization.
4. Transfer Learning leverages knowledge from pre-trained models to improve performance on related tasks, either by fine-tuning or feature extraction.

By incorporating these optimization strategies and meta-algorithms, deep learning models can achieve higher performance and robustness across various domains.